

Procés de visualització projectiu

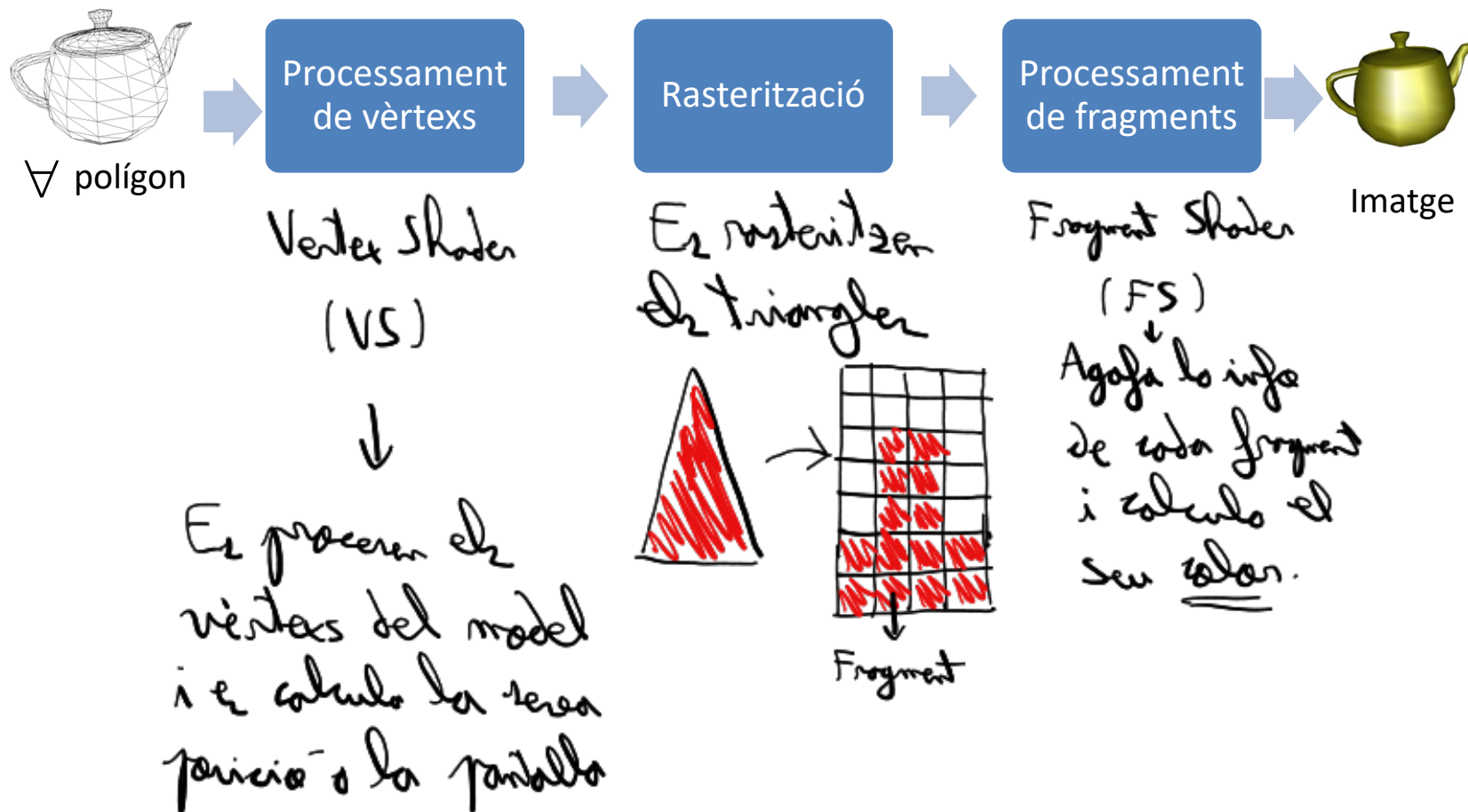
C. Andujar

Sep 2020

SISTEMES DE COORDENADES I

TRANSFORMACIONS GEOMÈTRIQUES

Paradigma projectiu simplificat



Simple.zip

• `pro` $\equiv$ `project`. On 'sources' sön ek fixen o ayafon
On 'Qt' diem d'war OpenGL i wid getz,
persönlich fixieren!

• $\Upsilon \equiv \text{code font}$

- glVertexAttribPointer (id, num-comp, type, stride, offset)

↓ ↓ ↓

0 3 GL_FLOAT

Vertex 3 floats



- glViewport(0, 0, width(), height())

Informació geomètrica al pipeline

vertices.push_back(QVector3D(**0, 1, 0**));

Coordenades d'un vèrtex

...

normals.push_back(QVector3D(**0, 0, 1**));

Components de la normal

setUniformValue("lightPosition", QVector3D(**0, 0, 0**));

Posició d'una llum

setUniformValue("lightDir", QVector3D(**0, 1, 0**));

Direcció d'una llum

Informació geomètrica al pipeline

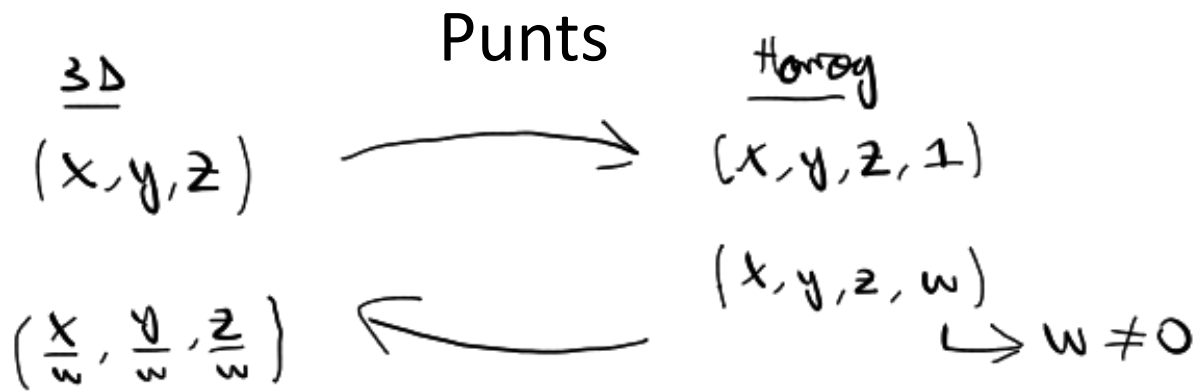
- Punts $\Rightarrow$ *vertex*
- Vectors
 - Normals (perpendiculars a la superfície)
 - Resta de vectors *Dir. llum*

Per un punt 3D hi ha ∞ homog.

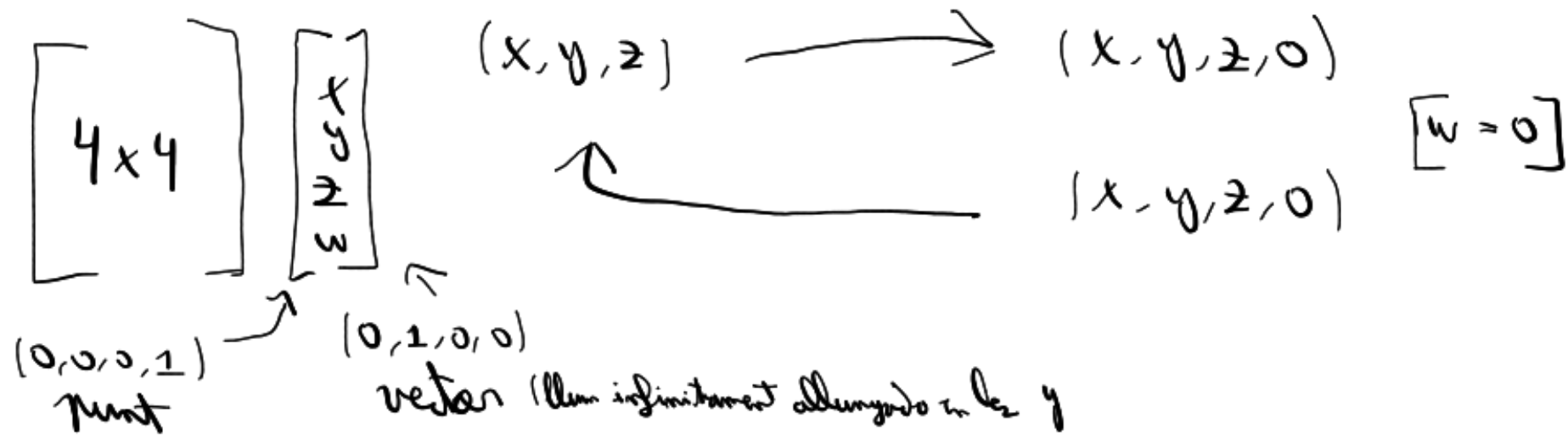
$(2, 3, 4, 1)$
 $\parallel$
 $(4, 6, 8, 2)$ $\searrow$ $(2, 3, 4)$

Coordenades homogènies (x, y, z, w)

$\downarrow$
 40 comp!



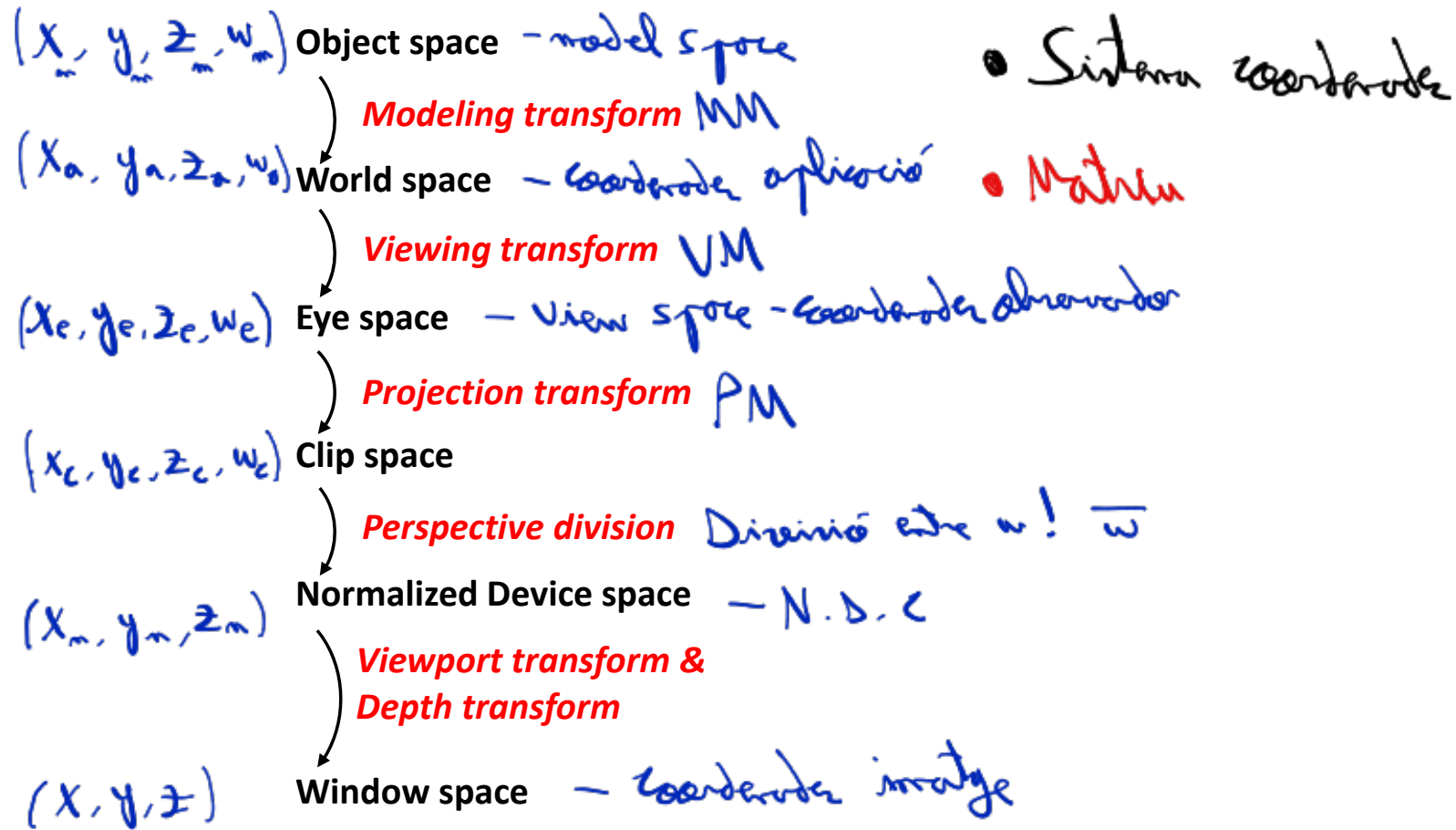
Vectors



TRANSFORMACIÓ DE PUNTS

(vèrtexs)

Sistemes de coordenades



VS les rep així!



Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

*Viewport transform &
Depth transform*

Window space

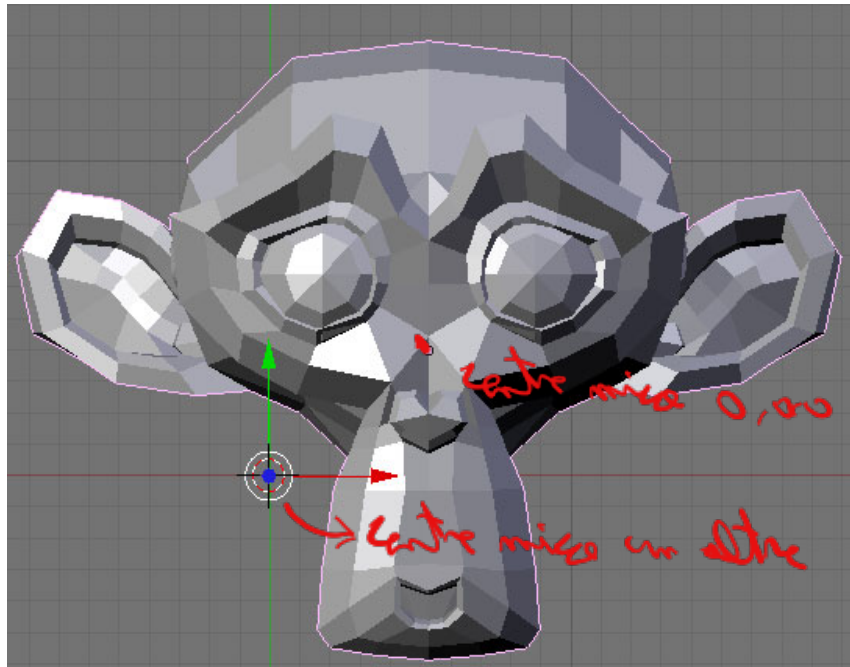
Object space (x_m, y_m, z_m, w_m)

Model space, SC del model, SC de l'objecte

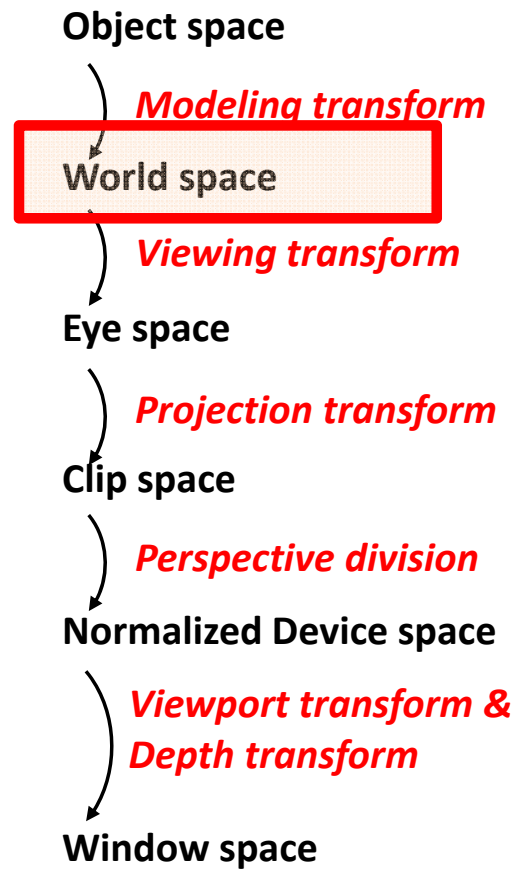
• es crea en Blender per

• SC utilitzat per modelar l'objecte.

• w_m normalment serà 1.0 (punts)



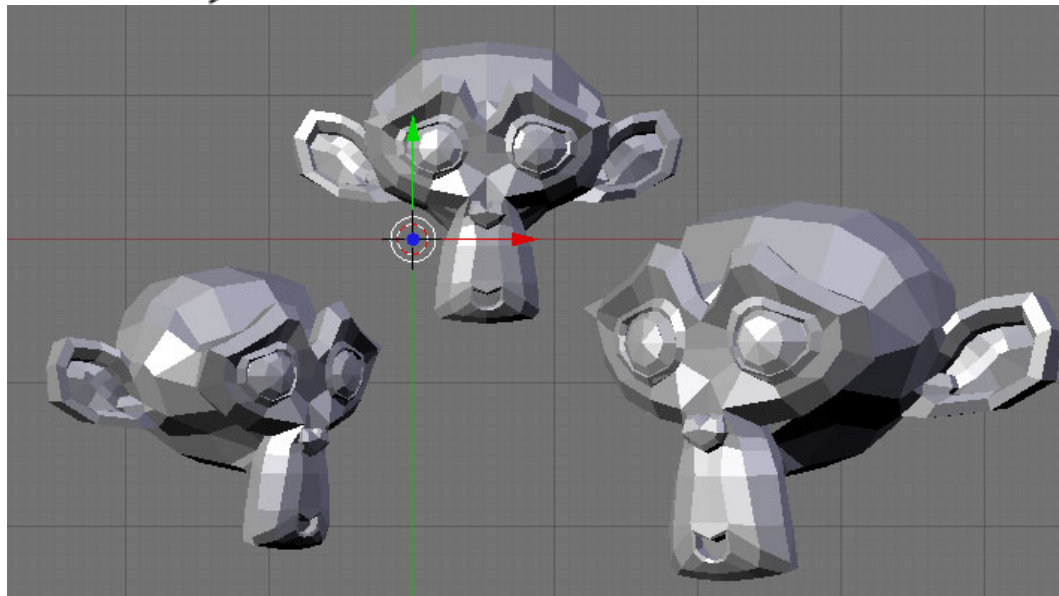
Sistemes de coordenades



World space (x_a, y_a, z_a, w_a)

SC de mon, SC de l'aplicació

- SC utilitzat per representar [l'escena]
- La transformació de modelat sovint preserva la component homogènia i per tant, w_a normalment serà (1.0 (punts))



Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

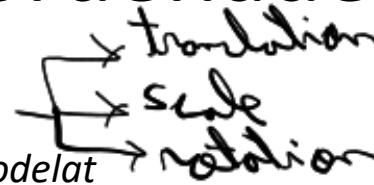
Normalized Device space

**Viewport transform &
Depth transform**

Window space

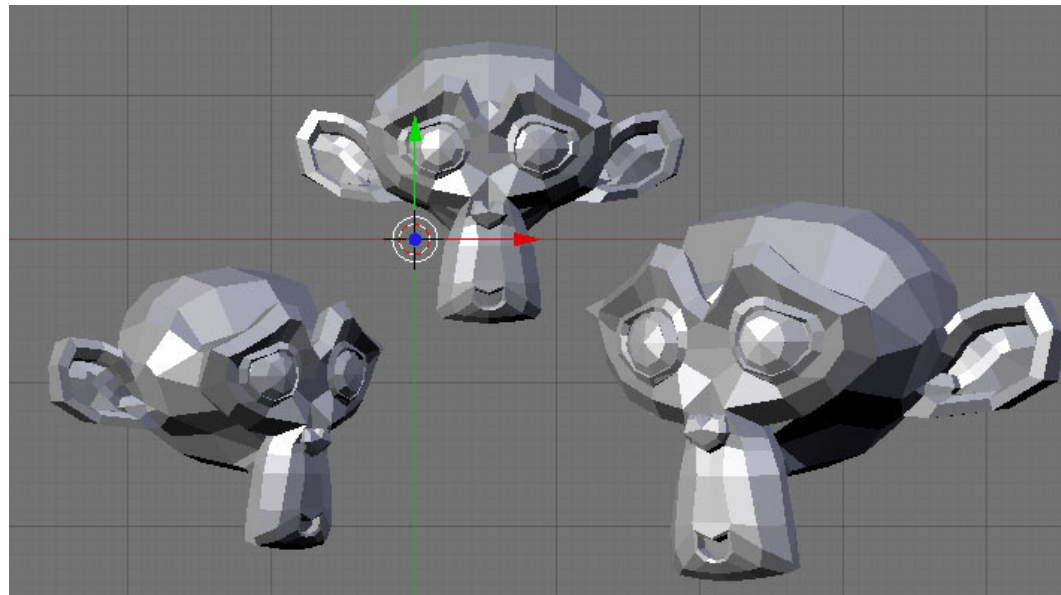
Modeling transform

Transformació de modelat



• Aquesta transformació situa cada instància d'un objecte en relació a l'escena.

• Sovint és la identitat, o una composició de translacions, rotacions i escalats



Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

**Viewport transform &
Depth transform**

Window space

Modeling transform

Transformació de modelat

$$\text{Translate}(t_x, t_y, t_z) \quad T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{Scale}(s_x, s_y, s_z) \quad T = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{Rotate}(a, x, y, z) \quad T = \begin{bmatrix} x^2d + c & xyd - zs & xzd + ys & 0 \\ yxd + zs & y^2d + c & yzd - xs & 0 \\ xzd - ys & yzd + xs & z^2d + c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$c = \cos(a)$, $s = \sin(a)$, $d = 1 - \cos(a)$

Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

**Viewport transform &
Depth transform**

Window space

Modeling transform

Transformació de modelat

$$\text{glRotate}*(\alpha, 1, 0, 0): \begin{matrix} \overbrace{\quad x \quad} \\ \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{matrix}$$

(no canvia x)

$$\text{glRotate}*(\alpha, 0, 1, 0): \begin{matrix} \overbrace{\quad y \quad} \\ \begin{bmatrix} \cos \alpha & 0 & \sin \alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \alpha & 0 & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{matrix}$$

(no canvia y)

$$\text{glRotate}*(\alpha, 0, 0, 1): \begin{matrix} \overbrace{\quad z \quad} \\ \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 & 0 \\ \sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{matrix}$$

(no canvia z)

Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

**Viewport transform &
Depth transform**

Window space

- Punts: matriu 4x4 multiplicada pel punt (l'ordre és important!)

$$\begin{bmatrix} x_a \\ y_a \\ z_a \\ w_a \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \begin{bmatrix} x_m \\ y_m \\ z_m \\ 1 \end{bmatrix}$$

model matrix *vertex*

1- Scale

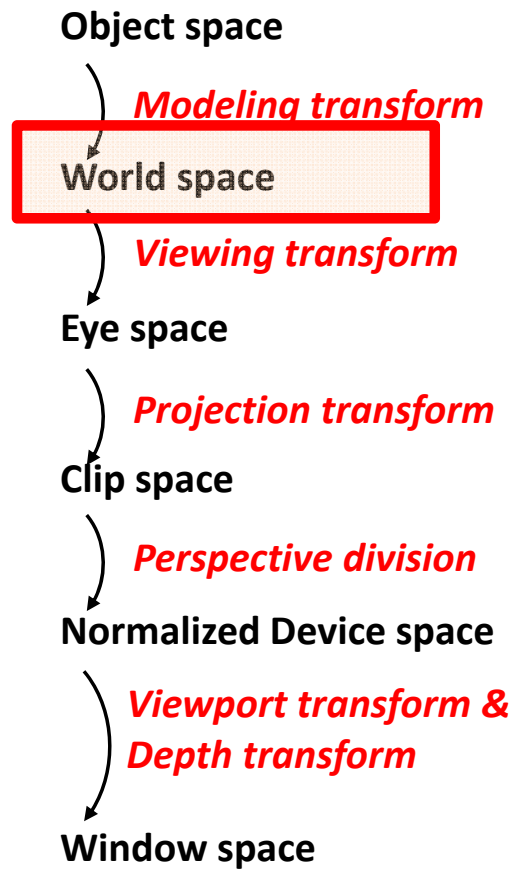
2- Rotate

3- Translation

$$\begin{matrix} (3) & (2) & (1) \\ T & \cdot & R & \cdot & S & = & MM \\ & & & & & & \text{(model matrix)} \end{matrix}$$

(Reye)

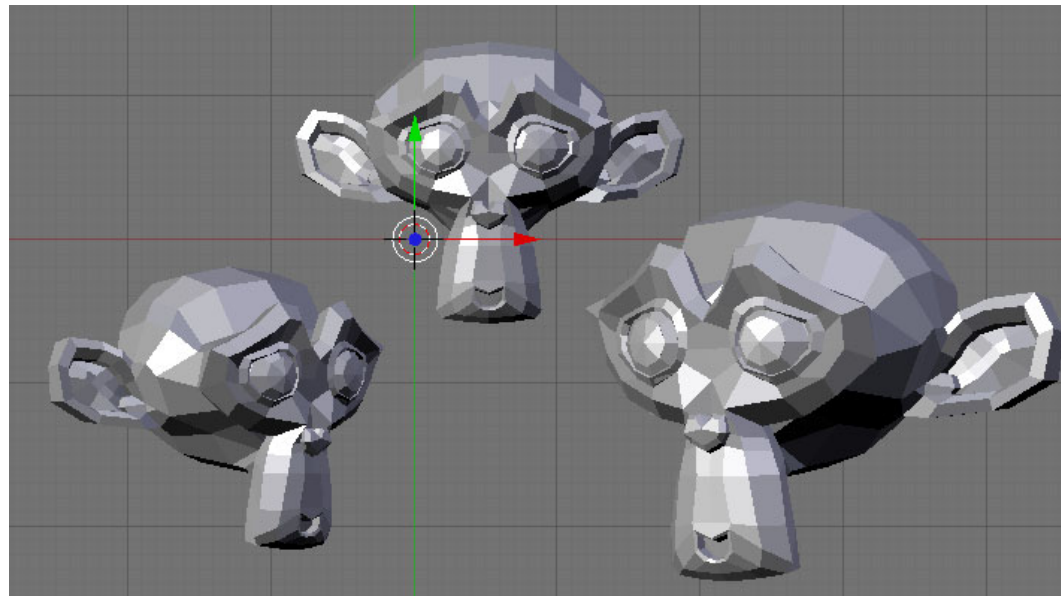
Sistemes de coordenades



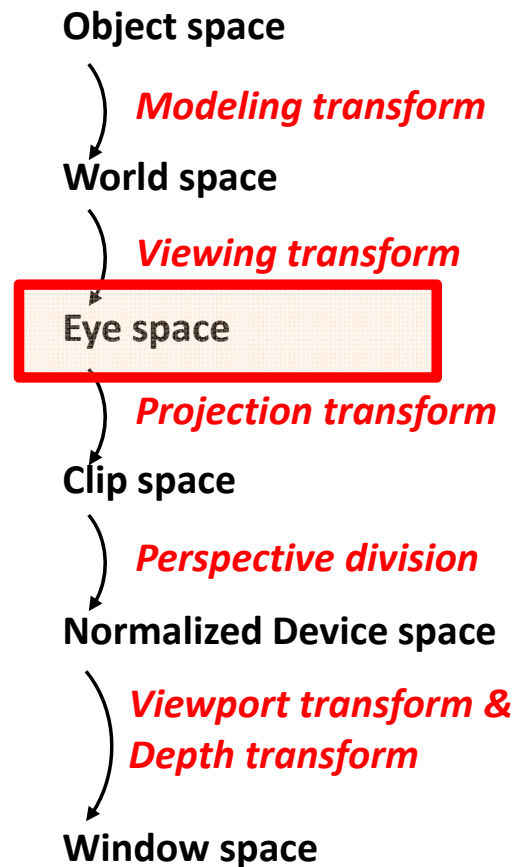
World space (x_a, y_a, z_a, w_a)
SC de mon, SC de l'aplicació

per a tots els objectes en l'escena!

- SC utilitzat per representar l'escena
- La transformació de modelat sovint preserva la component homogènia i per tant, w_a normalment serà 1.0 (punts)



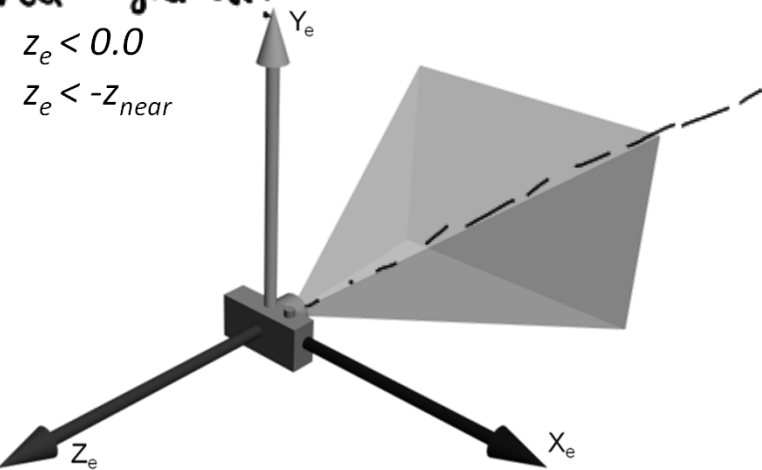
Sistemes de coordenades



Eye space (x_e, y_e, z_e, w_e)

SC de l'observador, SC de la càmera

- SC associat a la càmera
- La transformació de visualització sovint preserva la component homogènia i per tant, w_e normalment serà 1.0 (punts)
- Si la càmera és perspectiva, per tal que el punt sigui visible... *z del vert negativa!*



Propi sistema de coordenades!!!

Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

*Viewport transform &
Depth transform*

Window space

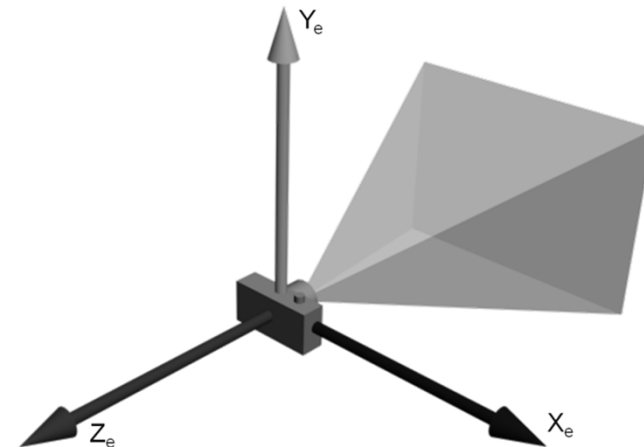
Viewing transform

Transformació de visualització, transformació de càmera

- Fa un canvi al sistema de referència de la càmera.
- Depèn de la posició i orientació de la càmera
- Sovint es defineix amb crides tipus lookAt, o amb una composició de translacions i rotacions:

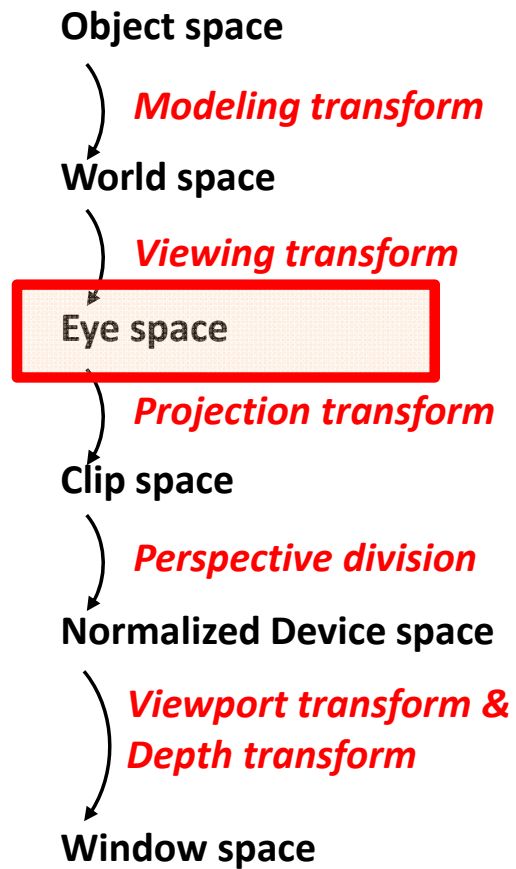
- $\text{lookAt}(\text{eye}, \text{target}, \text{up})$
- $T(0,0,-d) * R_z(-\varphi) * R_x(\Theta) * R_y(-\Psi) * T(-\text{VRP})$ (enter)

$$\begin{aligned} &T(0,0,-d) \\ &R(0,0,-\varphi) \\ &R(\Theta,0,0) \\ &R(0,\Psi,0) \\ &T(-\text{VRP}) \end{aligned}$$



(Reye)

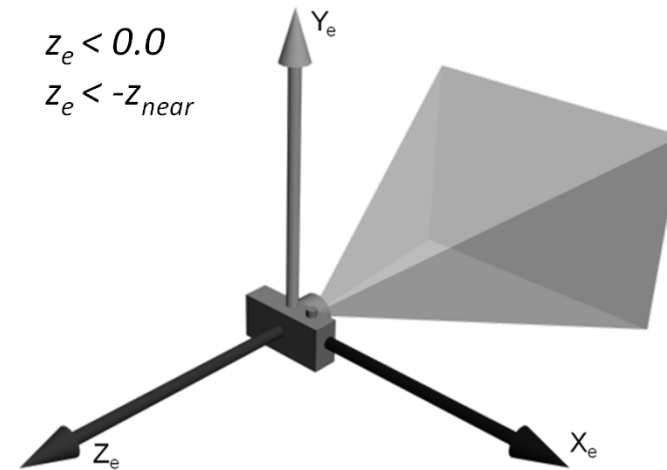
Sistemes de coordenades



Eye space (x_e, y_e, z_e, w_e)

SC de l'observador, SC de la càmera

- SC associat a la càmera
- La transformació de visualització sovint preserva la component homogènia i per tant, w_e normalment serà 1.0 (punts) o 0.0 (vectors)
- Si la càmera és perspectiva, per tal que el punt sigui visible...



Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

*Viewport transform &
Depth transform*

Window space

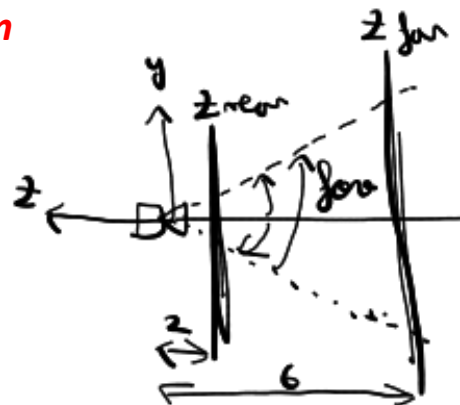
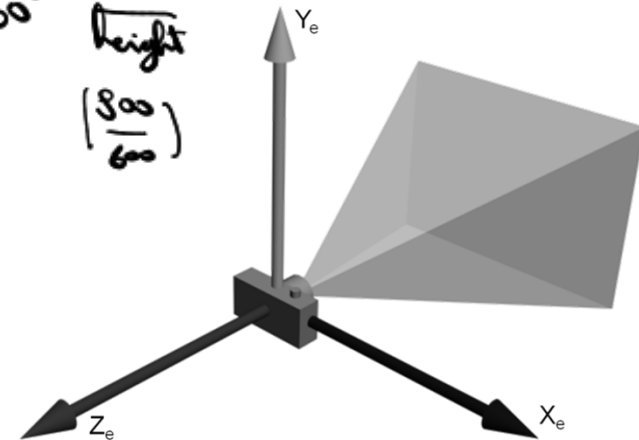
Projection transform

Transformació de projecció

- Depèn de la forma de la piràmide de visió i per tant del tipus de càmera (perspectiva, axonomètrica)
- Sovint es defineix amb crides del tipus **perspective**, **frustum**, **ortho**

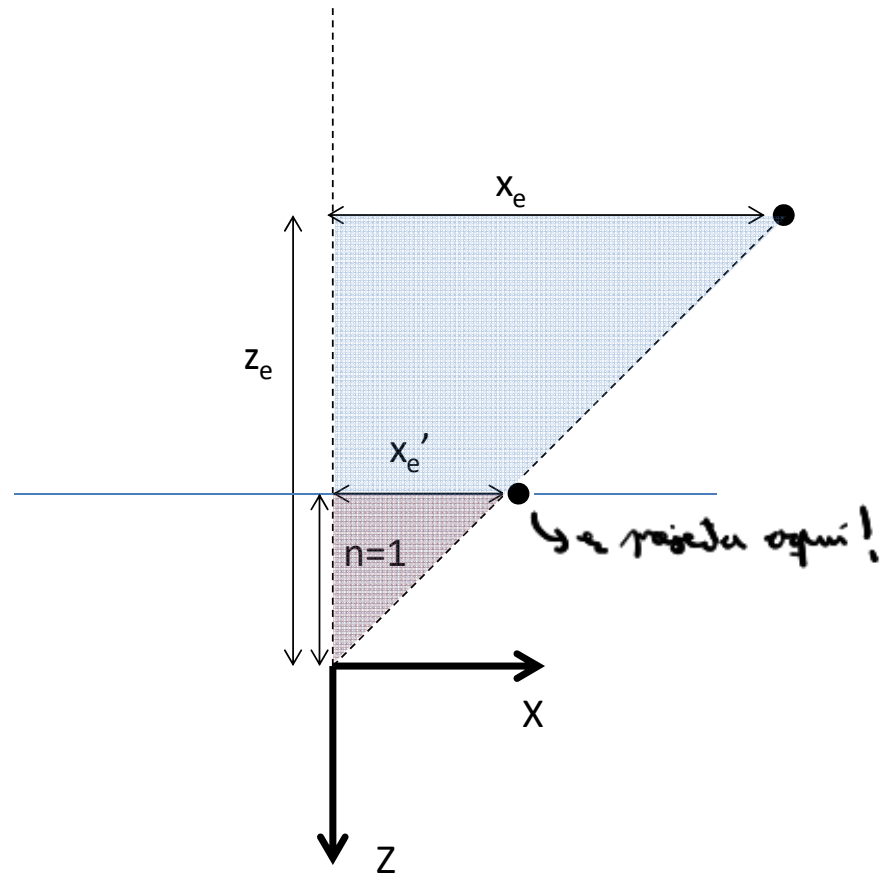
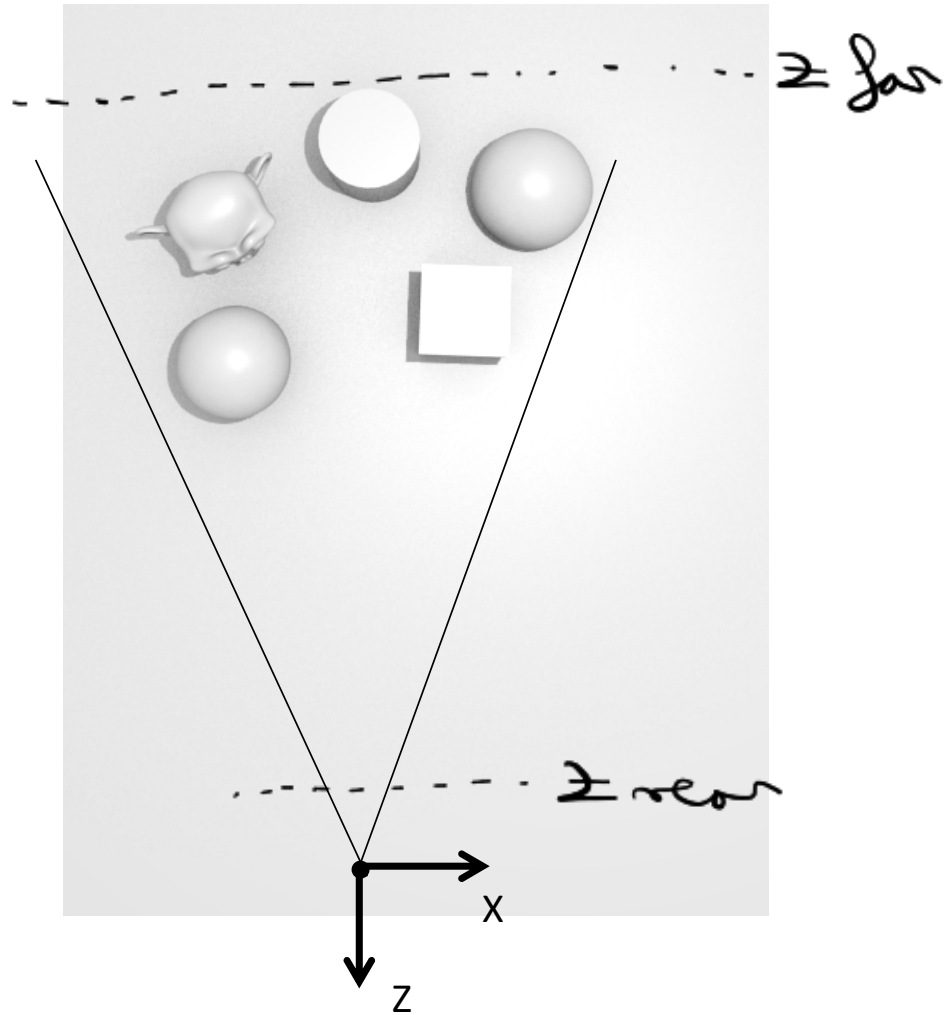
`perspective(fovy, aspect, near, far)`

60° width
height
(800/600)



z is negative!

Projecció i divisió per z_e



$$\frac{x_e'}{1} = \frac{x_e}{z_e}$$

perspective(90, 1, 1, 11);

$$\begin{bmatrix} \cot \frac{fovy}{2} & 0 & 0 & 0 \\ \frac{aspect}{\cot \frac{fovy}{2}} & 0 & 0 & 0 \\ 0 & \cot \frac{fovy}{2} & 0 & 0 \\ 0 & 0 & \frac{n+f}{n-f} & \frac{2*n*f}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix} \xrightarrow[n = 1, f = 11]{fovy=90, aspect=1} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1.2 & -2.2 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Pas eye space → clip space

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1.2 & -2.2 \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} x_e \\ y_e \\ z_e \\ 1 \end{bmatrix} = \begin{bmatrix} x_e \\ y_e \\ -1.2z_e - 2.2 \\ -z_e \end{bmatrix}$$

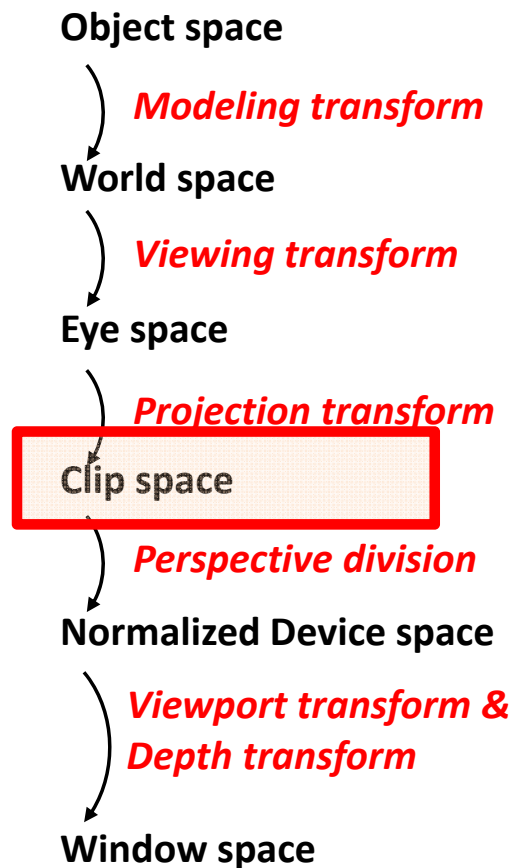
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1.2 & -2.2 \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} x_e \\ y_e \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} x_e \\ y_e \\ -1 \\ 1 \end{bmatrix}$$

Punt sobre el pla znear

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1.2 & -2.2 \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} x_e \\ y_e \\ -11 \\ 1 \end{bmatrix} = \begin{bmatrix} x_e \\ y_e \\ 11 \\ 11 \end{bmatrix}$$

Punt sobre el pla zfar

Sistemes de coordenades



Clip space (x_c, y_c, z_c, w_c)

Coordenades de clipping, coordenades de retallat

- Si un punt és interior al frustum, $(-1, 1)$

$$-w_c \leq x_c \leq w_c$$

$$-w_c \leq y_c \leq w_c$$

$$-w_c \leq z_c \leq w_c$$

- Si la càmera és perspectiva, llavors

$$w_c = -z_e$$

x, y, z entre $[-1, 1]$!!!

tanmateix

Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

*Viewport transform &
Depth transform*

Window space

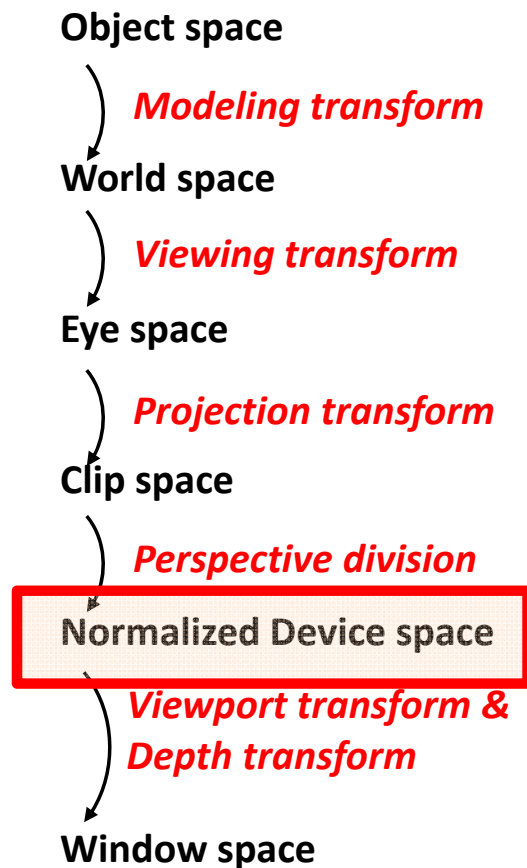
Perspective division

Divisió de perspectiva

- Simplement és el pas de coordenades homogènies a 3D
- Es divideix cada coordenada per la coord homogènia

$$(x_c, y_c, z_c, w_c) \rightarrow (x_c/w_c, y_c/w_c, z_c/w_c)$$

Sistemes de coordenades



Normalized device space (x_n, y_n, z_n)

NDC, cordenades normalitzades

- Si un punt és interior al frustum,

$$-1 \leq x_n \leq 1$$

$$-1 \leq y_n \leq 1$$

$$-1 \leq z_n \leq 1$$

- Els punts situats sobre znear tenen

$$z_n = -1$$

- Els punts situats sobre zfar tenen

$$z_f = +1$$

Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

Viewport transform & Depth transform

Window space

Viewport transformation

Transformació mon-dispositiu

$\{x, y\}$

- Depèn del viewport definit amb `glViewport(0, 0, width, height)`

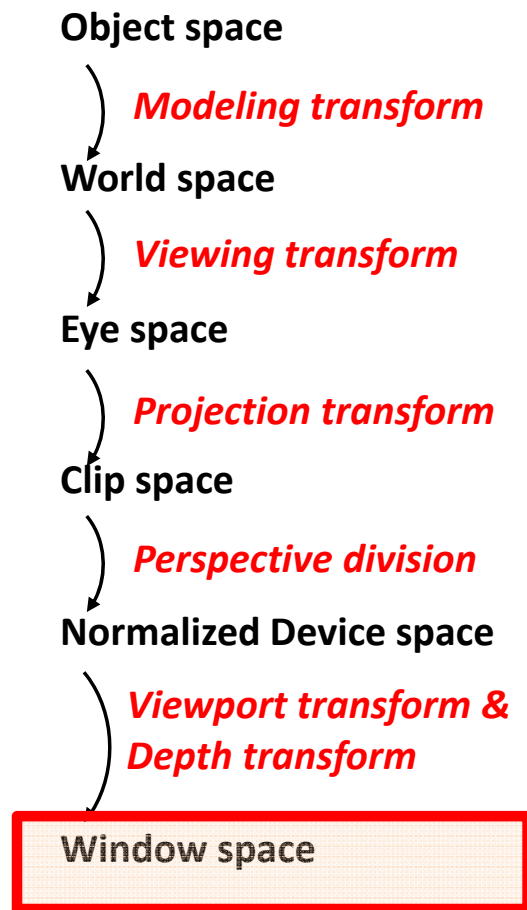
Depth range transformation

$\{z\}$

- Depèn de l'interval definit amb `glDepthRange` (per defecte $[0, 1]$)

$$\begin{aligned}
 x_n \in [-1, 1] &\xrightarrow{+1} [0, 2] \xrightarrow{/2} [0, 1] \xrightarrow{\cdot \text{width}} x \in [0, \text{width}] \\
 y_n &\dots \dots \dots \cdot \text{height} \\
 z_n \in [-1, 1] &\xrightarrow{+1} [0, 2] \xrightarrow{/2} [0, 1] \quad \Downarrow
 \end{aligned}$$

Sistemes de coordenades



Window space (x_d, y_d, z_d)

Cordenades de finestra, coordenades de dispositiu

- Si un punt és interior al frustum,

$$0 \leq x_d \leq w$$

$$0 \leq y_d \leq h$$

$$0 \leq z_d \leq 1$$

- Els punts situats sobre znear tenen

$$z_d = 0$$

- Els punts situats sobre zfar tenen

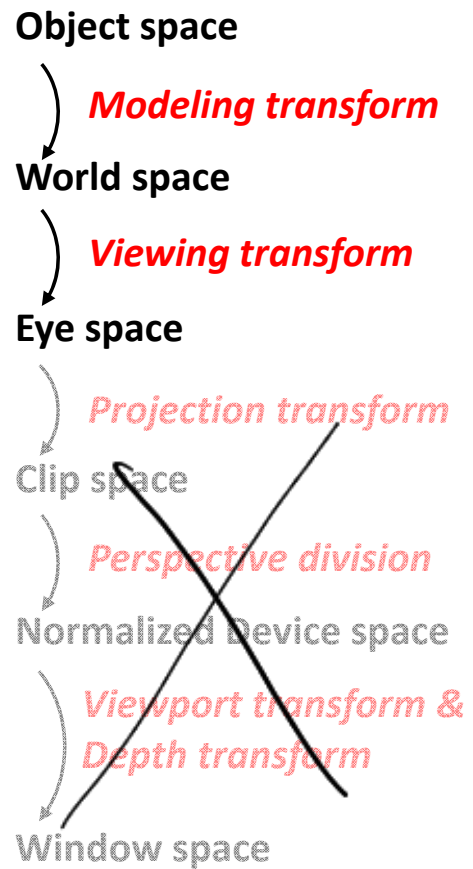
$$z_d = 1$$

- Quan més endavant es generin fragments,

$$gl_FragCoord.w = 1/w_c = -1/z_e$$

TRANSFORMACIÓ DE VECTORS (EXCEPTE NORMALS)

Sistemas de coordenadas



Em iluminação não precisamos o eye !!

Sistemas de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

*Viewport transform &
Depth transform*

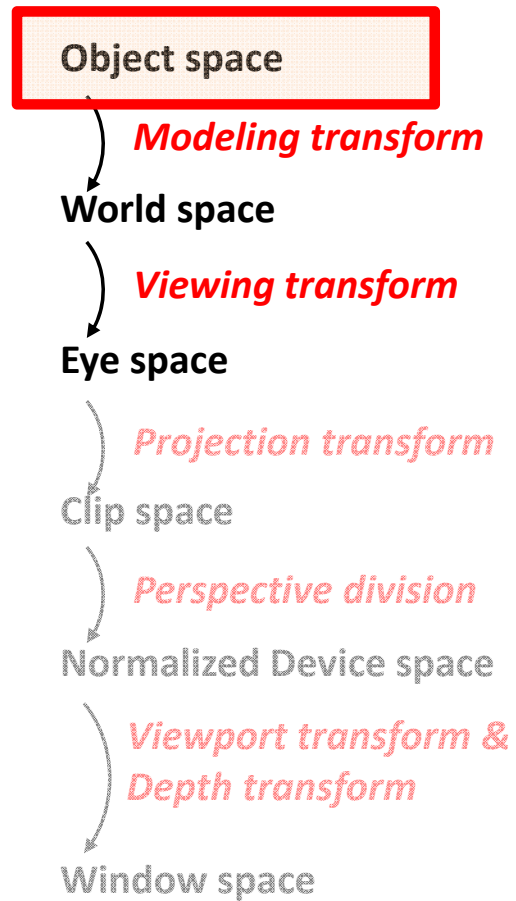
Window space

$$\vec{v} = Q - P \quad (\text{der punkt})$$

origin i fi

$$\vec{v}_w = f(Q) - f(P)$$

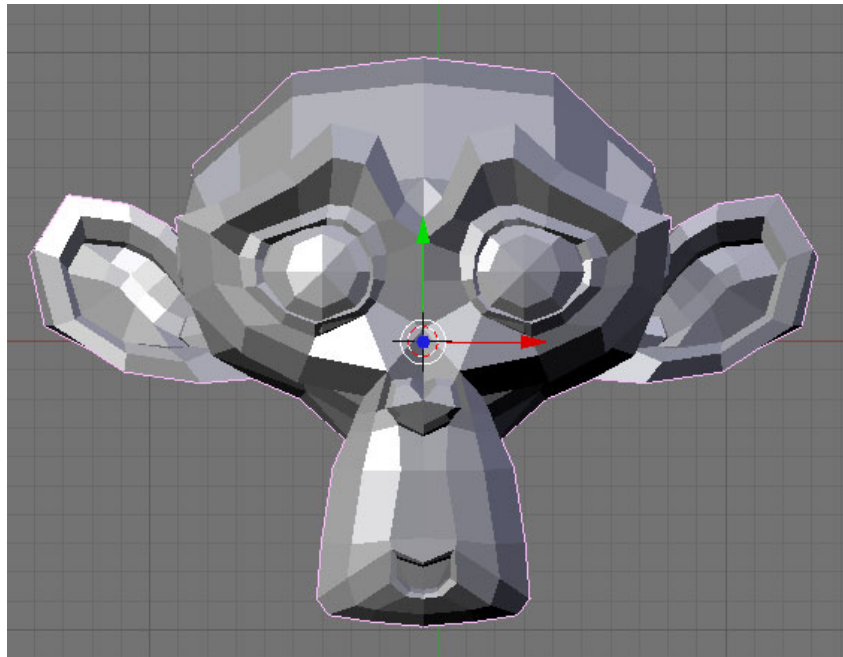
Sistemes de coordenades



Object space (x_m, y_m, z_m, w_m)

Model space, SC del model, SC de l'objecte

- SC utilitzat per modelar l'objecte.
- w_m normalment serà 1.0 (punts) o 0.0 (vectors)



Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

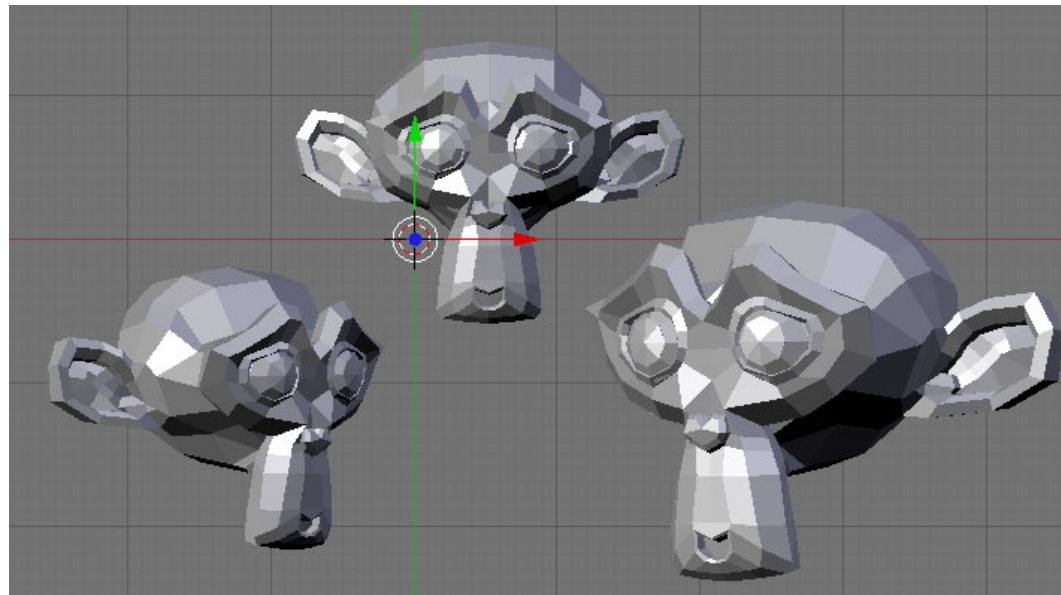
**Viewport transform &
Depth transform**

Window space

Modeling transform

Transformació de modelat

- Aquesta transformació situa cada instància d'un objecte en relació a l'escena.
- Sovint és la identitat, o una composició de translacions, rotacions i escalats



Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

**Viewport transform &
Depth transform**

Window space

Modeling transform

Transformació de modelat

$$\text{translate}(t_x, t_y, t_z) \quad T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{scale}(s_x, s_y, s_z) \quad T = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{rotate}(a, x, y, z) \quad T = \begin{bmatrix} x^2d + c & xyd - zs & xzd + ys & 0 \\ yxd + zs & y^2d + c & yzd - xs & 0 \\ xzd - ys & yzd + xs & z^2d + c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$c = \cos(a)$, $s = \sin(a)$, $d = 1 - \cos(a)$

Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

**Viewport transform &
Depth transform**

Window space

Modeling transform

Transformació de modelat

$$\text{glRotate}*(\alpha, 1, 0, 0): \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha & 0 \\ 0 & \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{glRotate}*(\alpha, 0, 1, 0): \begin{bmatrix} \cos\alpha & 0 & \sin\alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\alpha & 0 & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{glRotate}*(\alpha, 0, 0, 1): \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 & 0 \\ \sin\alpha & \cos\alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Sistemes de coordenades

Object space

Modeling transform

World space

Viewing transform

Eye space

Projection transform

Clip space

Perspective division

Normalized Device space

**Viewport transform &
Depth transform**

Window space

- Punts, vectors: matriu 4x4 multiplicada pel punt/vector

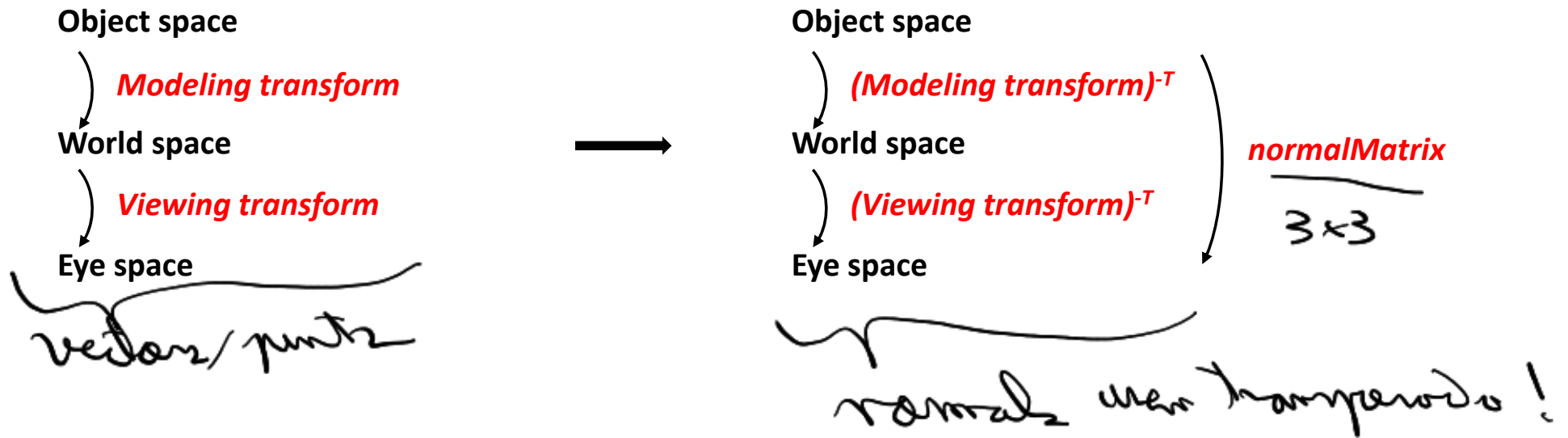
$$\begin{bmatrix} x_a \\ y_a \\ z_a \\ w_a \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \begin{bmatrix} x_m \\ y_m \\ z_m \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x_a \\ y_a \\ z_a \\ w_a \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \begin{bmatrix} x_m \\ y_m \\ z_m \\ 0 \end{bmatrix}$$

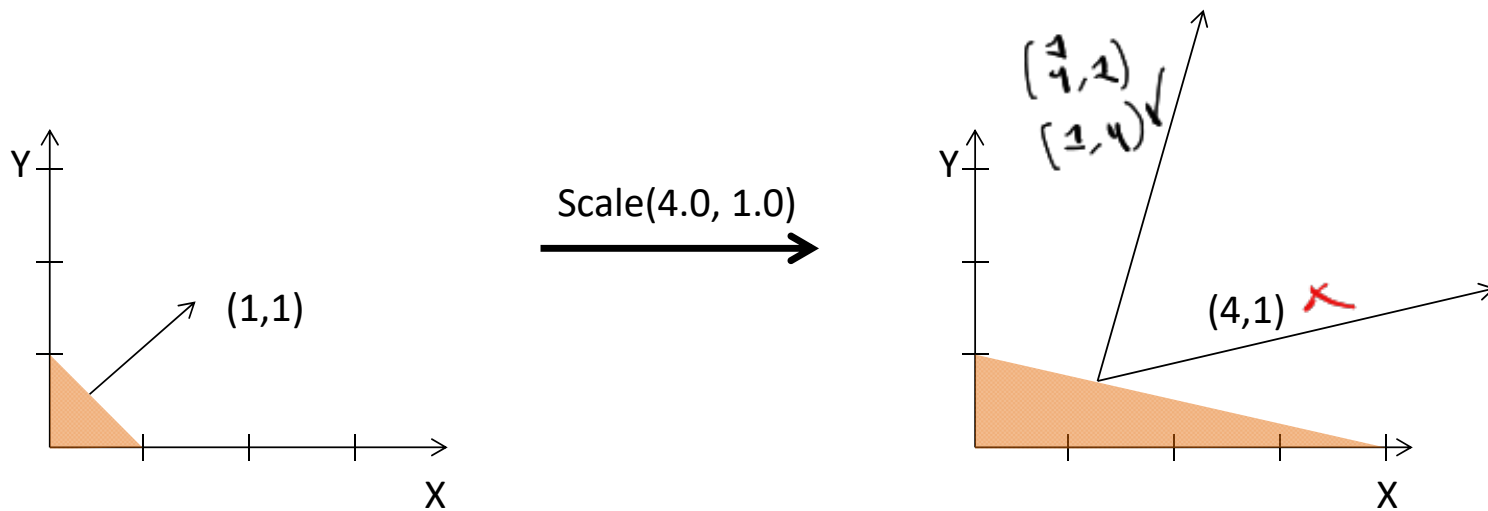
TRANSFORMACIÓ NORMALS

For moving entre object i eye

Sistemas de coordenades



Sistemas de coordenades



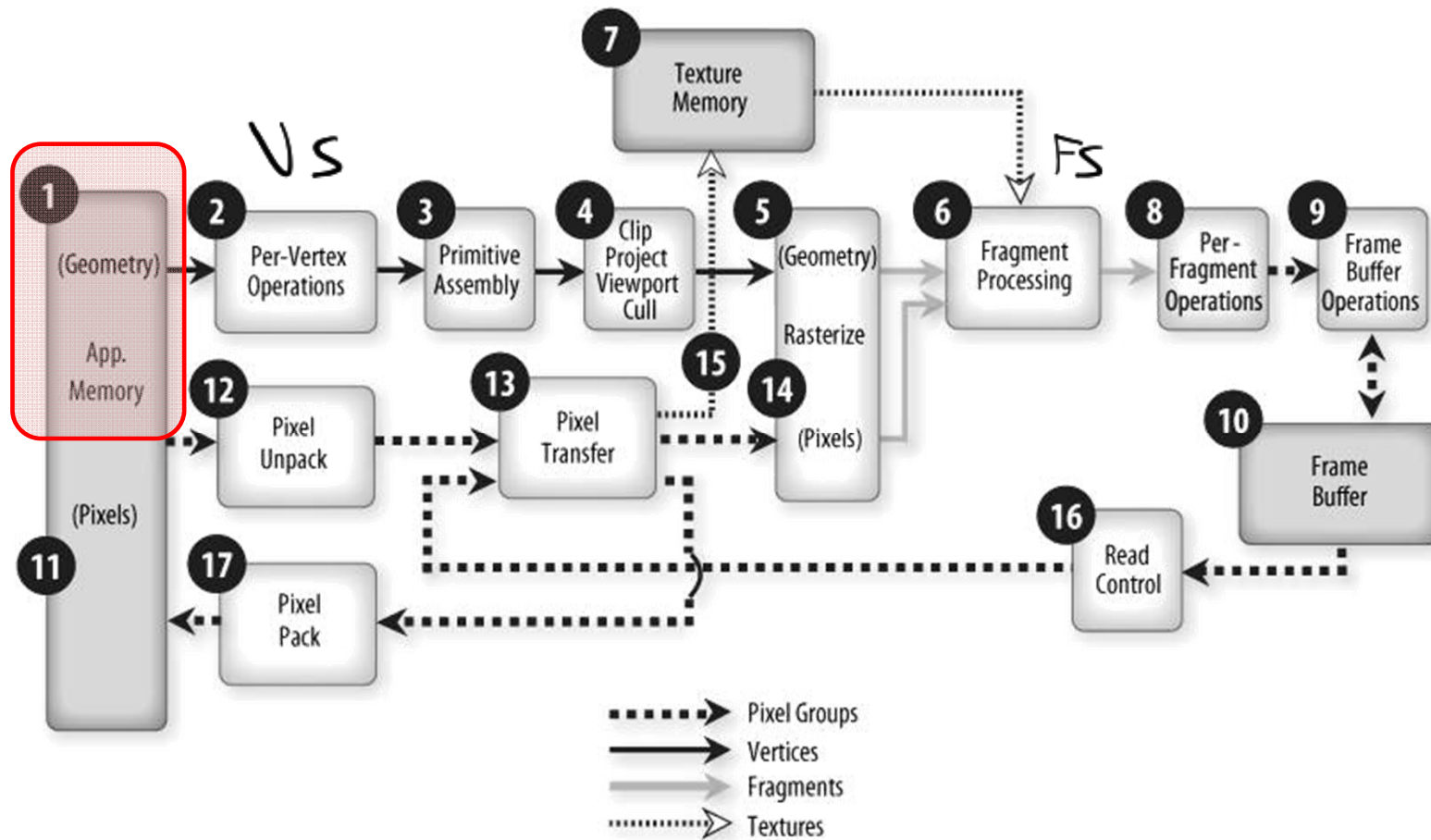
Transformació de normals



$$N \perp V \equiv N \cdot V = 0 \equiv \underset{1 \times 3}{N^T} \cdot \underset{3 \times 1}{V} = 0$$

PIPELINE OPENGL

Pipeline OpenGL



Pipeline OpenGL

1. Dibuix de primitives

Les primitives (punts, línies, polígons...) es poden pintar:

- 1 • Vèrtex a vèrtex: *glBegin, glVertex, glEnd* (compatibility profile) (mode immediat immediat per cada vert)
- 2 • Vertex array object: *glDrawArrays, glDrawElements...*

(mode immediate)

Exemple (*compatibility*)

```
glBegin(GL_POLYGON);  
  glNormal3f(nx0, ny0, nz0);  
  glVertex3f(x0, y0, z0);  
  glNormal3f(nx1, ny1, nz1);  
  glVertex3f(x1, y1, z1);  
  ...  
glEnd();
```

Coords i normals en
object space

Es calculen cada cop que cal pintar
B

Exemple (*compatibility/core*)

```
// createBuffers
glm::vec3 Vertices[...]; // Tres vèrtexs amb X, Y i Z
Vertices[0] = glm::vec3(-1.0, -1.0, 0.0);
...
glGenVertexArrays(1, &VAO);
glBindVertexArray(VAO);

glGenBuffers(1, &VBO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(Vertices), Vertices, GL_STATIC_DRAW);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
glEnableVertexAttribArray(0);

glBindVertexArray(0);

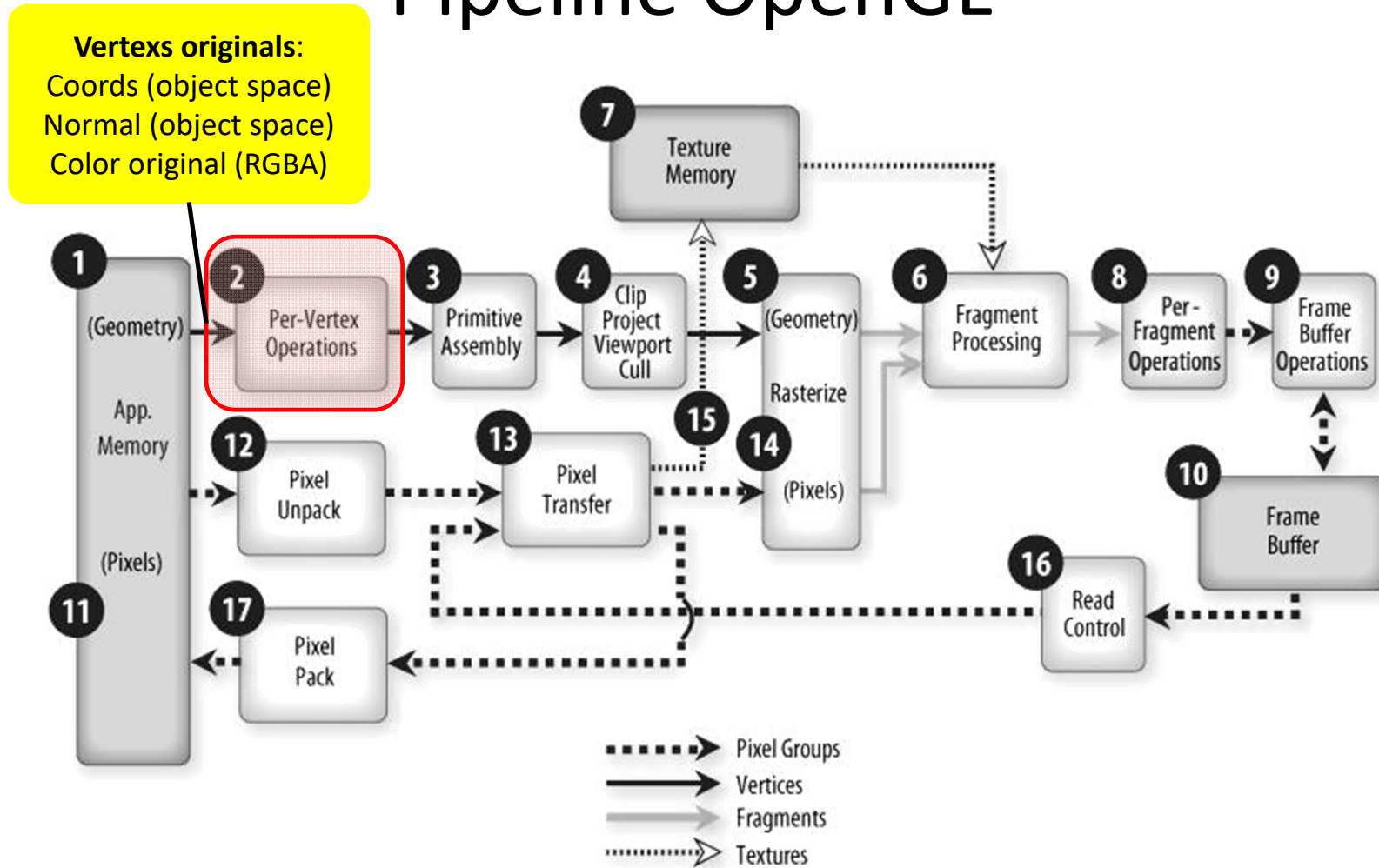
// paintGL
glBindVertexArray(VAO);
glDrawArrays(GL_TRIANGLES, 0, numVerts);
glBindVertexArray(0);
```

Coords i normals en
object space

Es calcula només un cop ↓ ↓

→ active pipeline

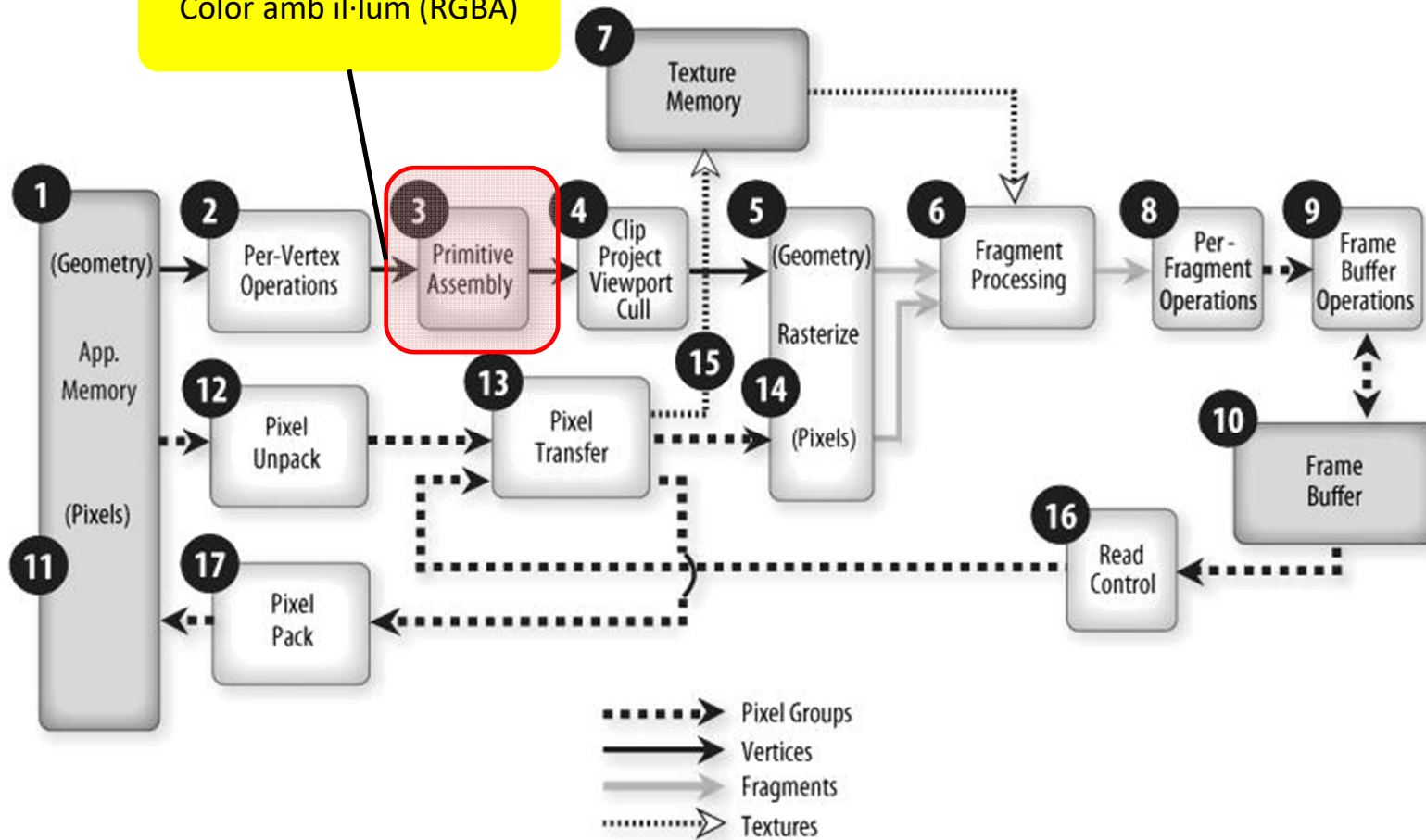
Pipeline OpenGL



Pipeline OpenGL

2. Per-vertex operations (transform de normals i vèrtex) + (il·luminació s: val)
- Es transformen els vèrtexs (modelview i projection).
 - Es transformen les normals (trasposta de l'inversa de la submatriu 3x3 de la modelview) i es pot calcular la il·luminació del vèrtex.

Vertices transformats:
Coords (clip space)
Color amb il·lum (RGBA)



Pipeline OpenGL

3. Primitive assembly

- Els vèrtexs s'agrupen formant primitives.
- Cada primitiva requereix un clippling diferent.

Pipeline OpenGL - glDrawArrays

```
glDrawArrays(mode, first, count);
```

vertex			
	x	y	z
V0	1.0	0.0	0.0
V1			
V2			
V3			
V4			
V5			
V6			
V7			
...			

GL_POINTS (Cada vértex um ponto diferente)

GL_LINES (Cada par de vértices são linhas) v_0 com v_1
 v_2 com v_3
...

GL_TRIANGLES (Cada 3 vértices formam um triângulo) v_0, v_1, v_2
 v_3, v_4, v_5

GL_TRIANGLE_STRIP (tira de um triângulo) v_0, v_1, v_2
 v_1, v_2, v_3
 v_2, v_3, v_4
...

el man per
va repetin vertices!

Pipeline OpenGL - glDrawArrays

(int
uint)

glDrawElements(mode, count, type, indices);

vertex

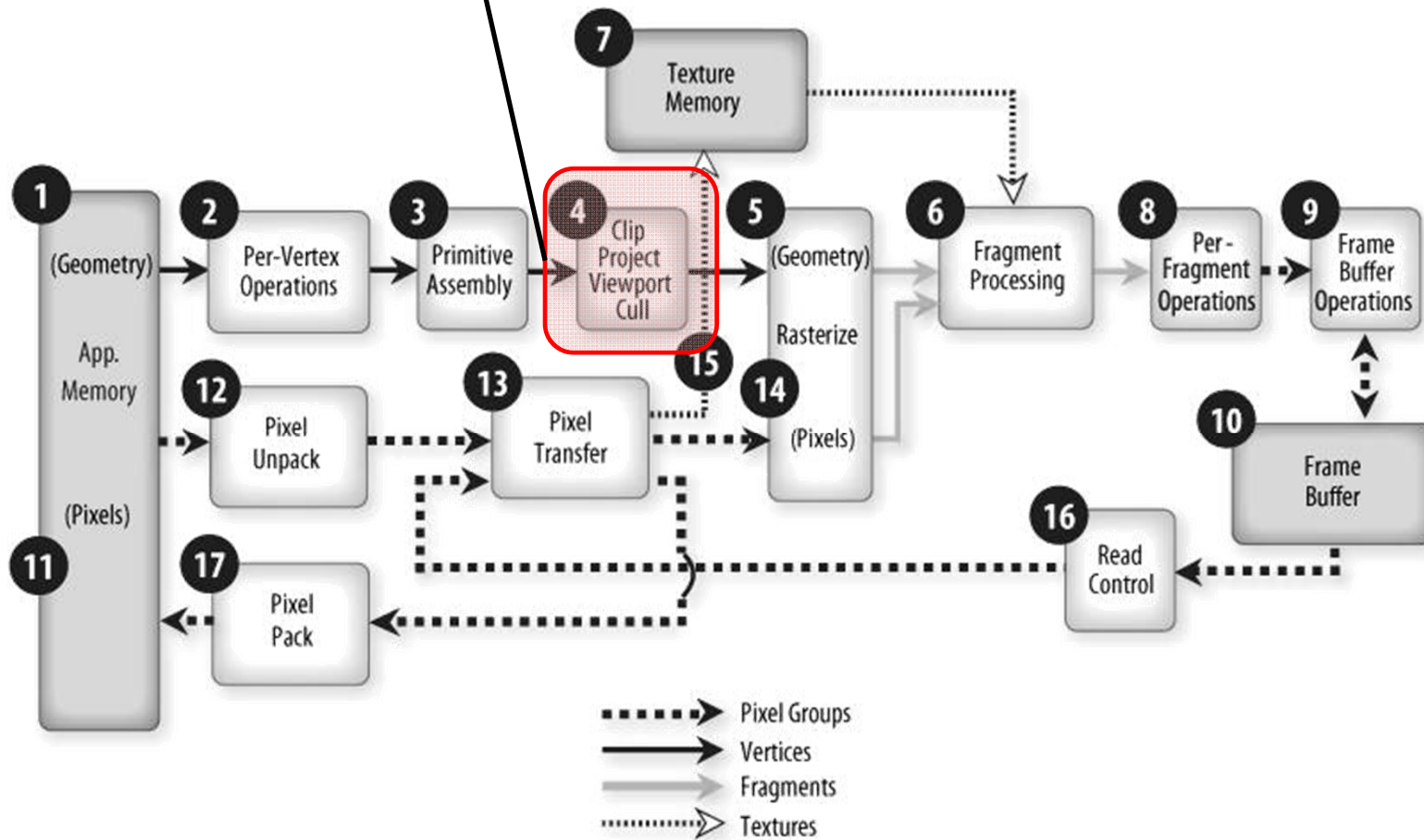
indices

	x	y	z		id
V0	1.0	0.0	0.0	i0	0
V1				i1	3
V2				i2	5
V3				i3	2
V4				i4	1
V5				i5	6
V6				i6	
V7				i7	
	...	...	...		...

GL_TRIANGLES

GL_TRIANGLE_STRIP

**Primitives transformed:
(clip space) i il·luminades**



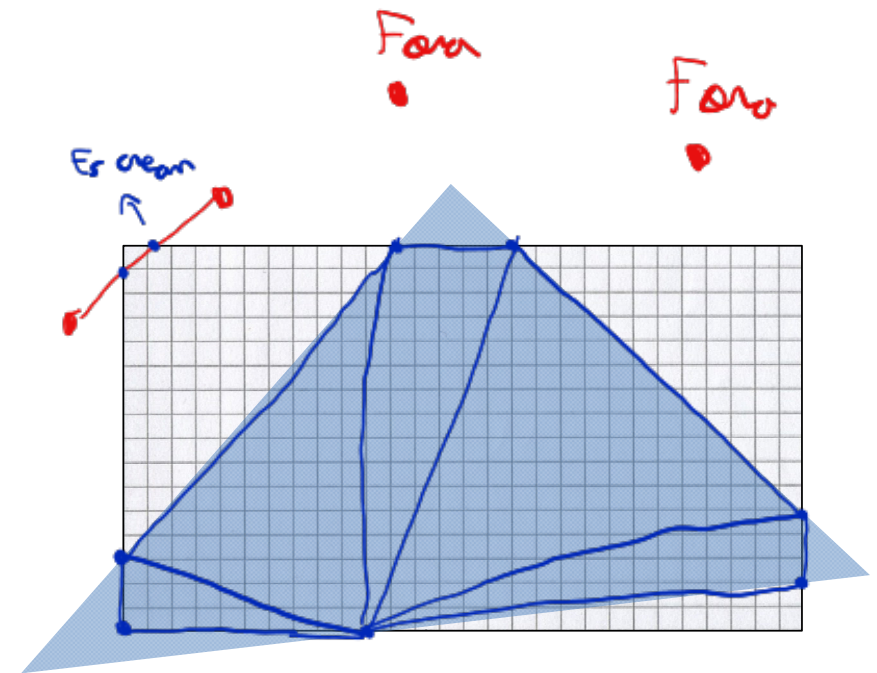
Pipeline OpenGL

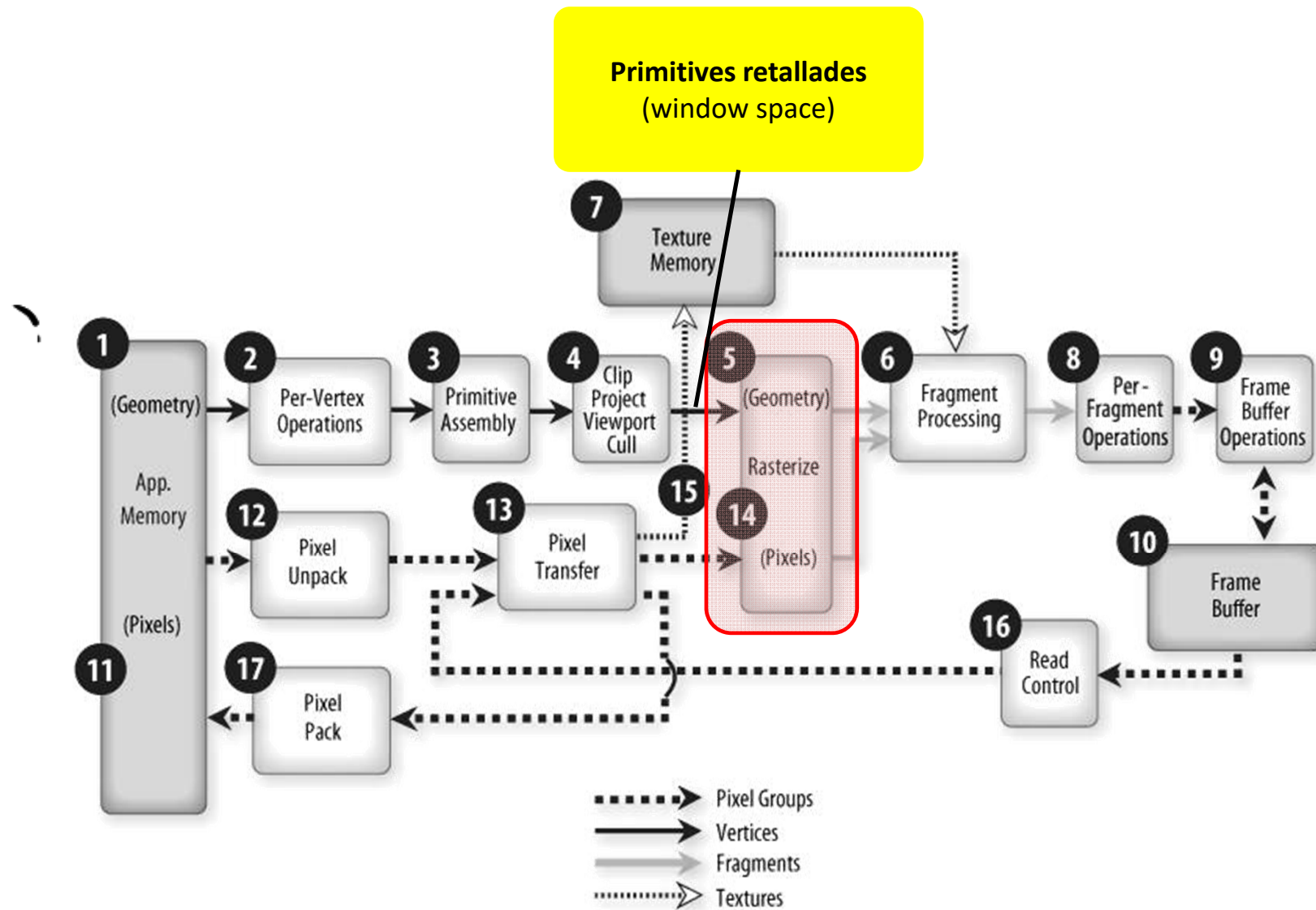
4. Primitive processing

- Clipping (retallat) a la piràmide de visió. (Si està fora piràmide veïa l'eliminar)
- Divisió de perspectiva: es divideix (x, y, z) per w
- Viewport & depth transform $\rightarrow$ window space
- Backface culling – `glEnable(GL_CULL_FACE)`

(eliminar cares que la seu normal no mira
o ho cobreix)

$$\begin{matrix} \rightarrow -w_c \leq x_c \leq w_c \\ y_c \\ z_c \end{matrix}$$





Pipeline OpenGL

OpenGL mira si
el centro del fragment
conté primitiva!

5. Rasterització

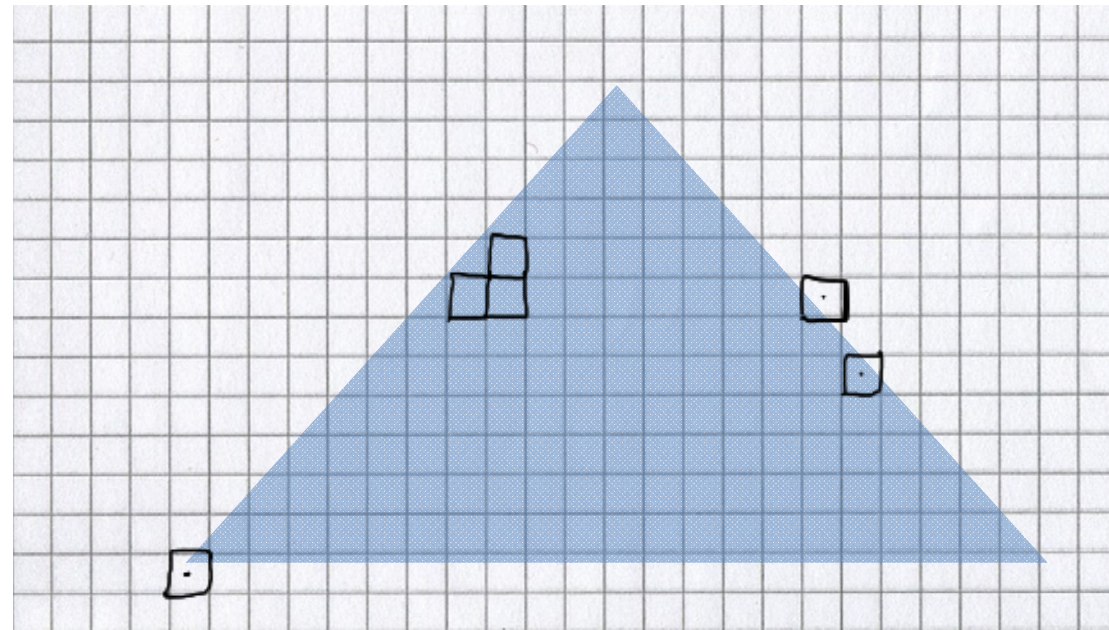
Generació dels fragments corresponents a la primitiva retallada.

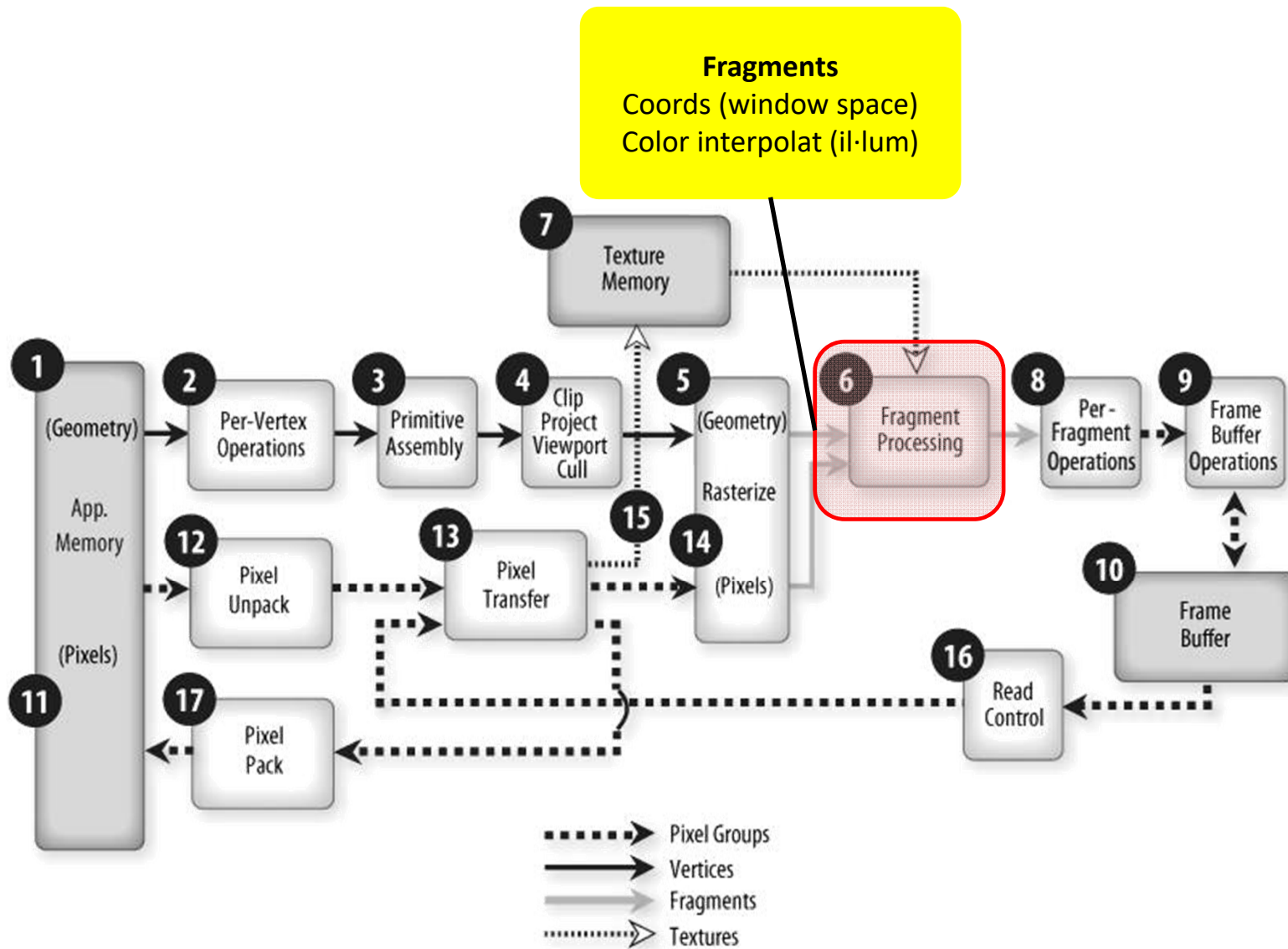
Cada fragment té usualment diversos atributs:

- Coordenades (window space)
- Color (interpolat)
- Coordenades de textura (interpolades)

→ generem Normal, color, coordenades
a cada □

□ → fragments " "
Fa border detectant fragments



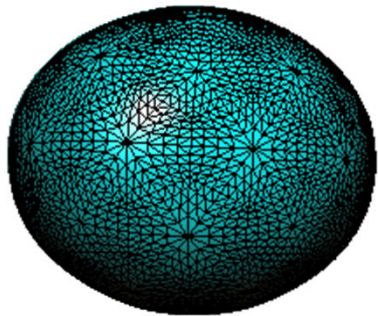
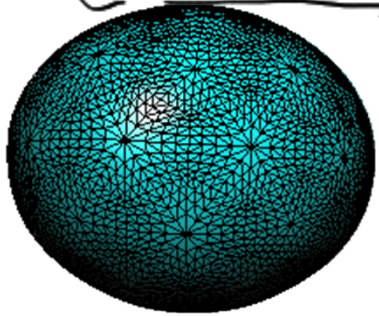


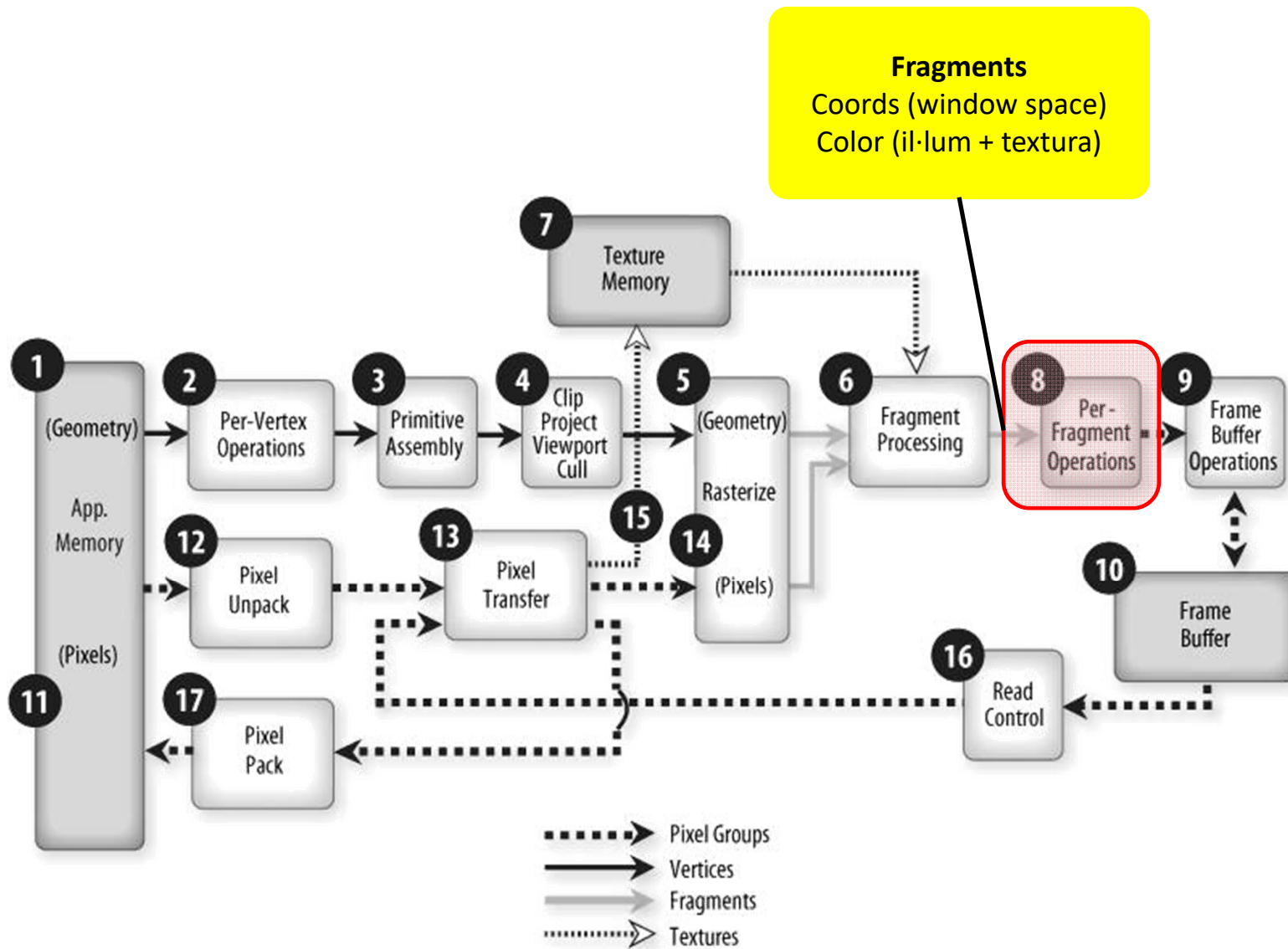
Pipeline OpenGL

1 Fragment shader

6. Fragment processing (“shading”)

- Càlcul del color del fragment (texture mapping, il·lum per fragment, etc).





Pipeline OpenGL

8. Per-fragment operations (“raster operations”)

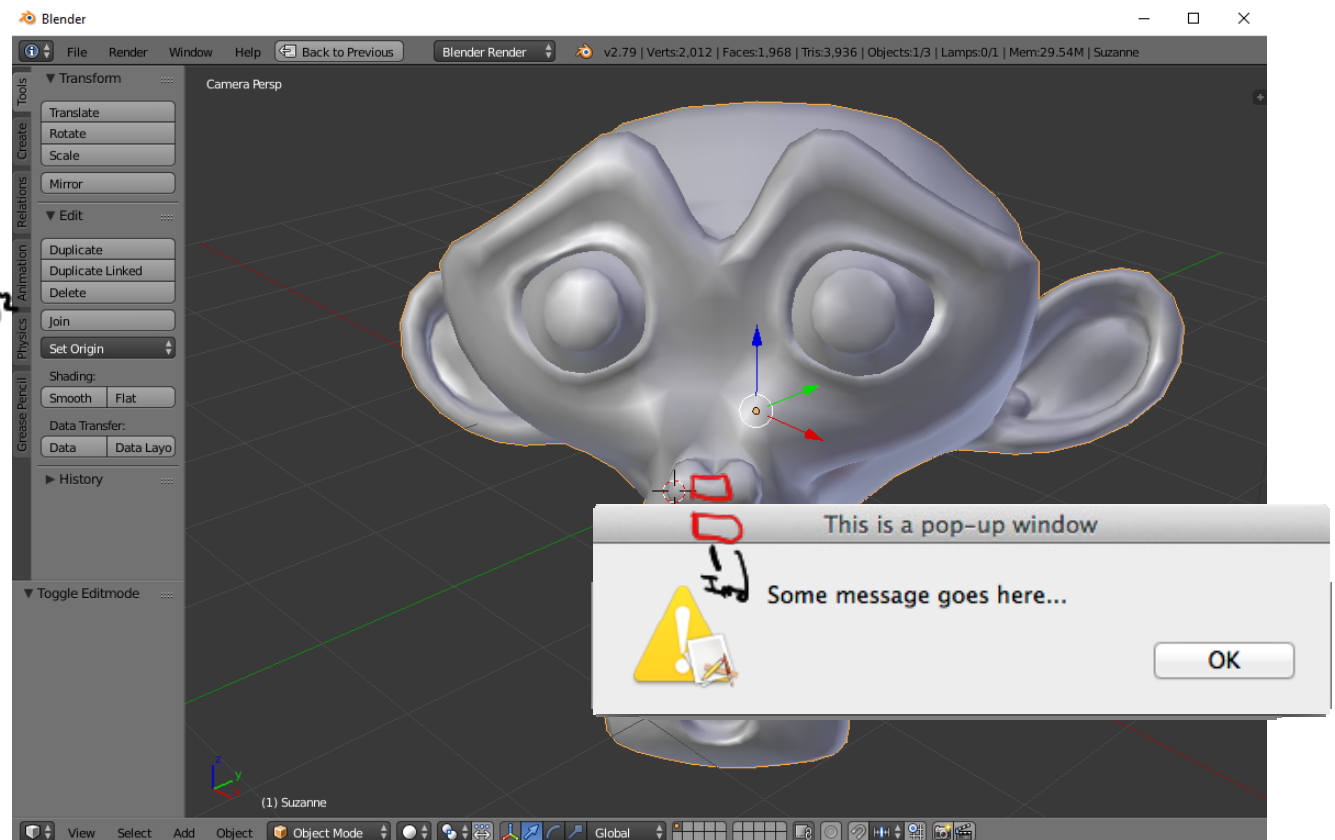
- Pixel ownership test
- *Scissor test*
- *Alpha test*
- Stencil test
- Depth test (test Z-buffer)
- Blending
- *Dithering*
- *Logical Ops* (*glLogicOp*)

Pipeline OpenGL

8. Per-fragment operations (“raster operations”)

- **Pixel ownership test**
- *Scissor test*
- *Alpha test*
- Stencil test
- Depth test (test Z-buffer)
- Blending
- *Dithering*
- *Logical Ops (glLogicOp)*

*No define it
selon le frag
shader d'autre app*

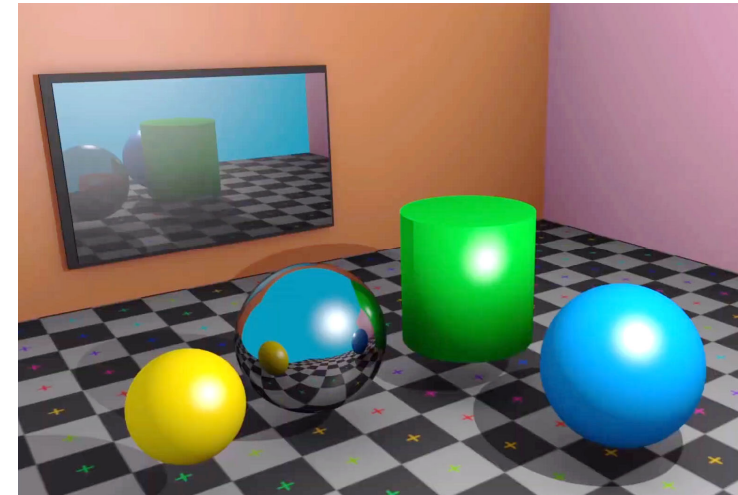


Pipeline OpenGL

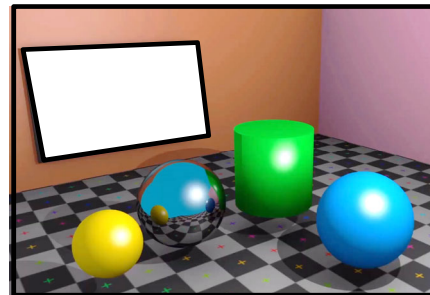
8. Per-fragment operations (“raster operations”)

- Pixel ownership test
- *Scissor test*
- *Alpha test*
- **Stencil test**
- Depth test (test Z-buffer)
- Blending
- *Dithering*
- *Logical Ops (glLogicOp)*

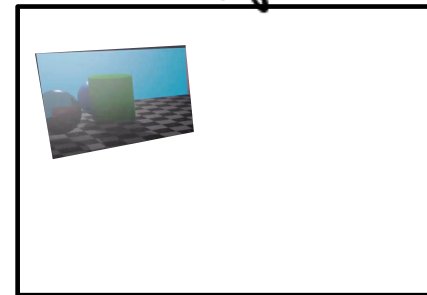
Test per limitar rasterització
a una part de la finestra
Open GL utilitzant una màscara!



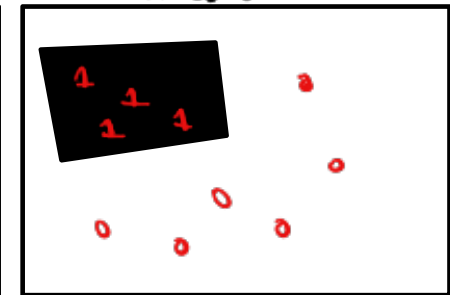
1 render



2 + rendering



mask



```
glEnable(GL_STENCIL_TEST); (actiu)
```

```
glStencilFunc(comparació, valorRef, mask)  
GL_ALWAYS, GL_EQUAL, ...
```

```
glStencilOp(fail, zfail, zpass)  
GL_KEEP, GL_ZERO, GL_INCR, GL_REPLACE
```

Pipeline OpenGL

8. Per-fragment operations (“raster operations”)

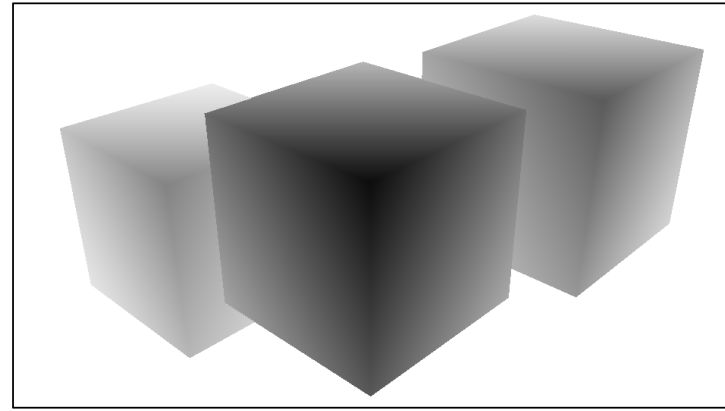
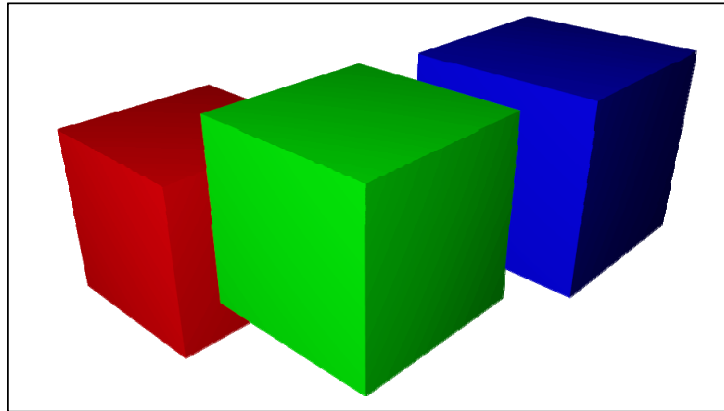
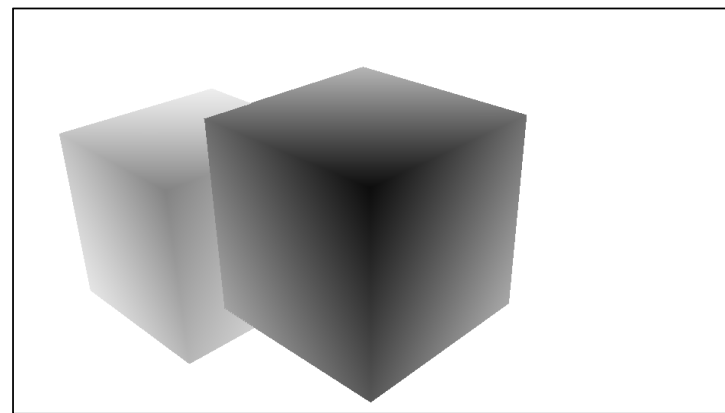
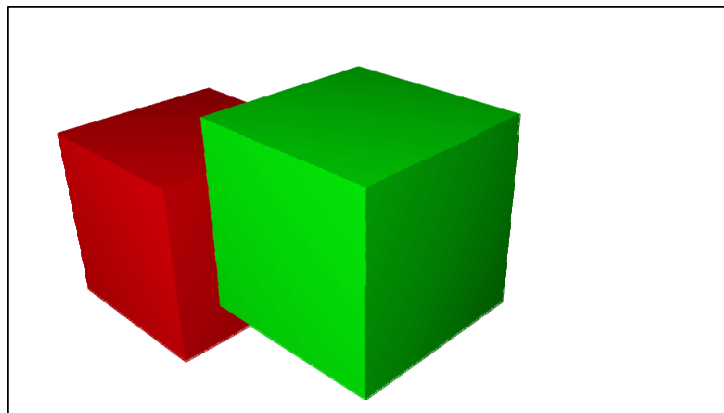
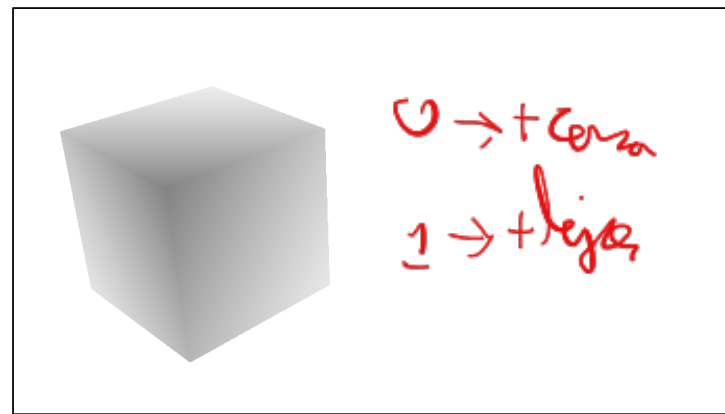
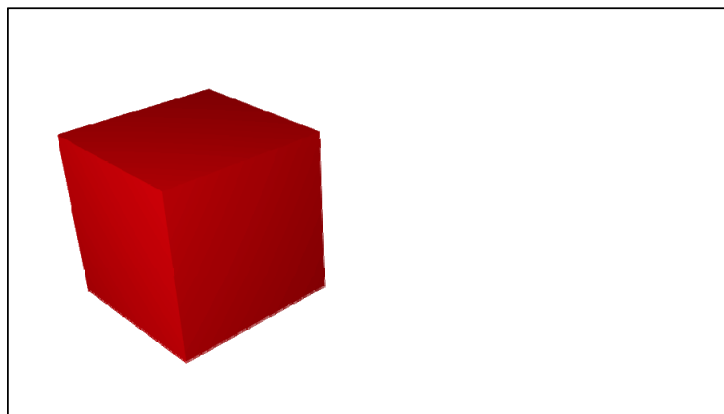
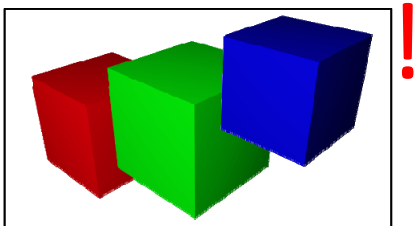
- Pixel ownership test
- *Scissor test*
- *Alpha test*
- Stencil test
- **Depth test (test Z-buffer)**
- Blending
- *Dithering*
- *Logical Ops (glLogicOp)*



```
glEnable(GL_DEPTH_TEST);
```

```
glDepthFunc(GL_LESS);
```

```
GL_LESS, GL_LEQUAL, GL_GREATER, ...
```



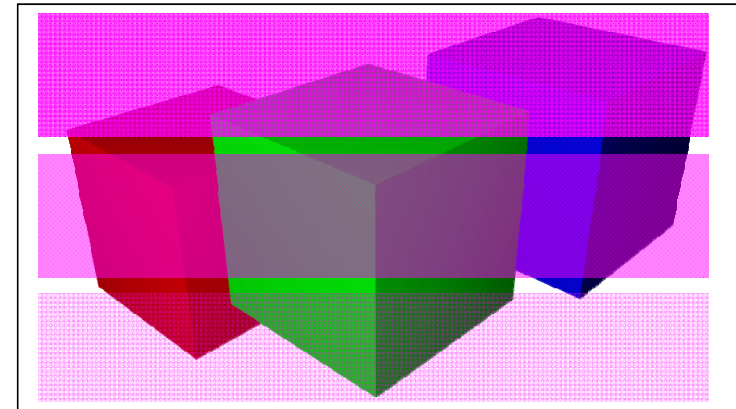
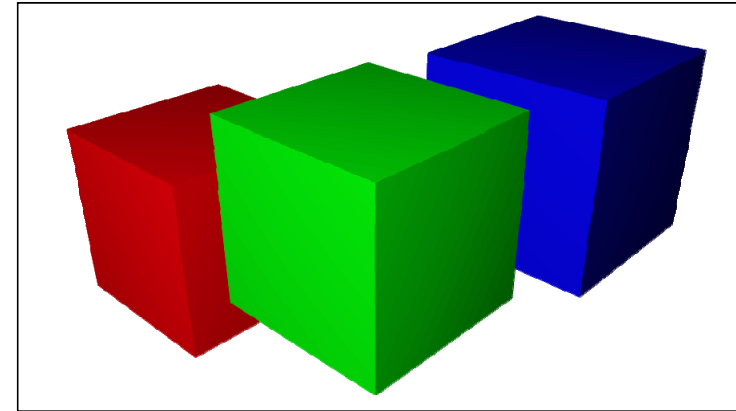
Pipeline OpenGL

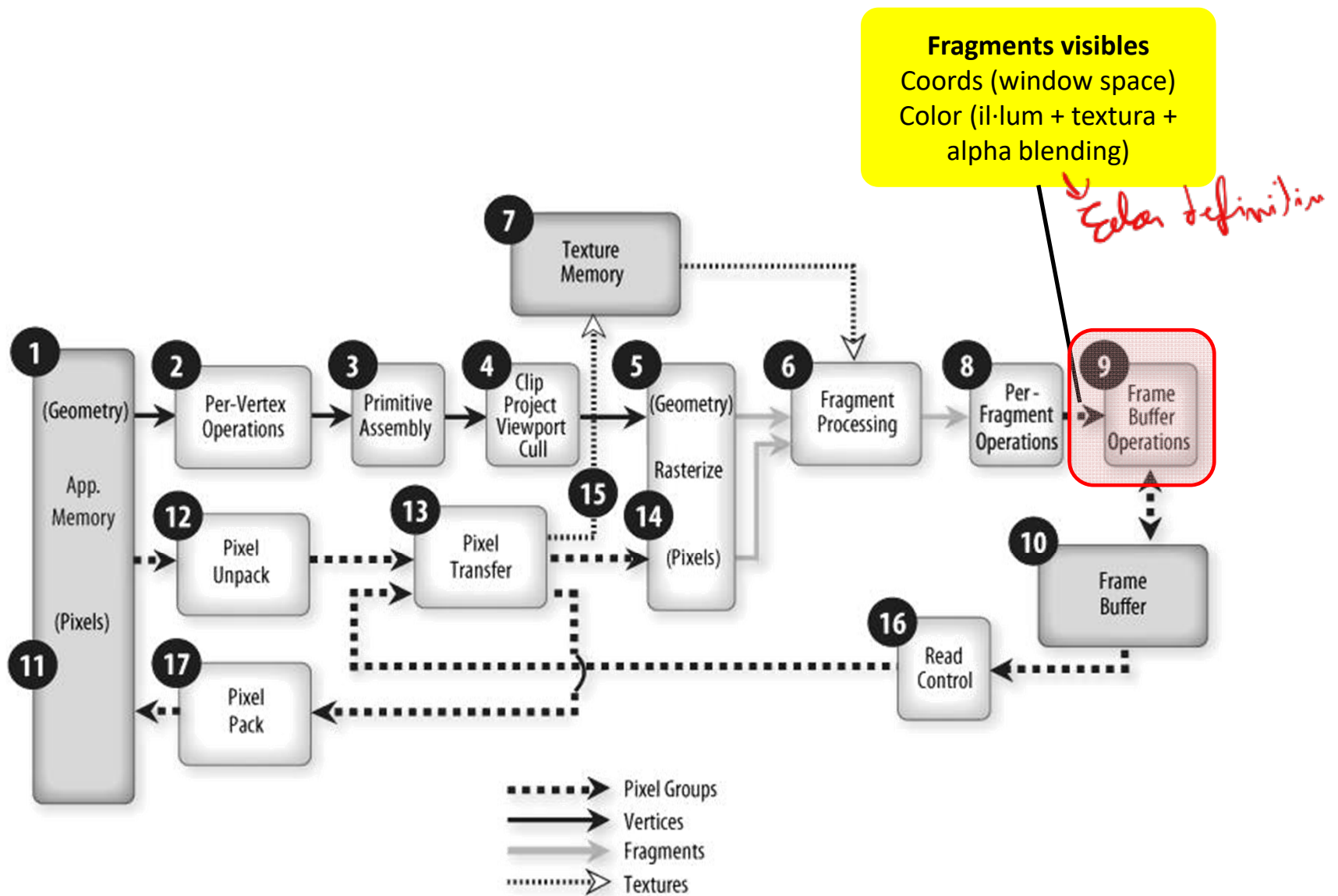
8. Per-fragment operations (“raster operations”)

- Pixel ownership test
- *Scissor test*
- *Alpha test*
- Stencil test
- Depth test (test Z-buffer)
- **Blending** → *translucidez (permite ver objetos que hay detrás)*
- *Dithering*
- *Logical Ops* (*glLogicOp*)

```
glEnable(GL_BLEND);
```

```
glBlendFunc(...);
```





Pipeline OpenGL

9. Frame buffer operations

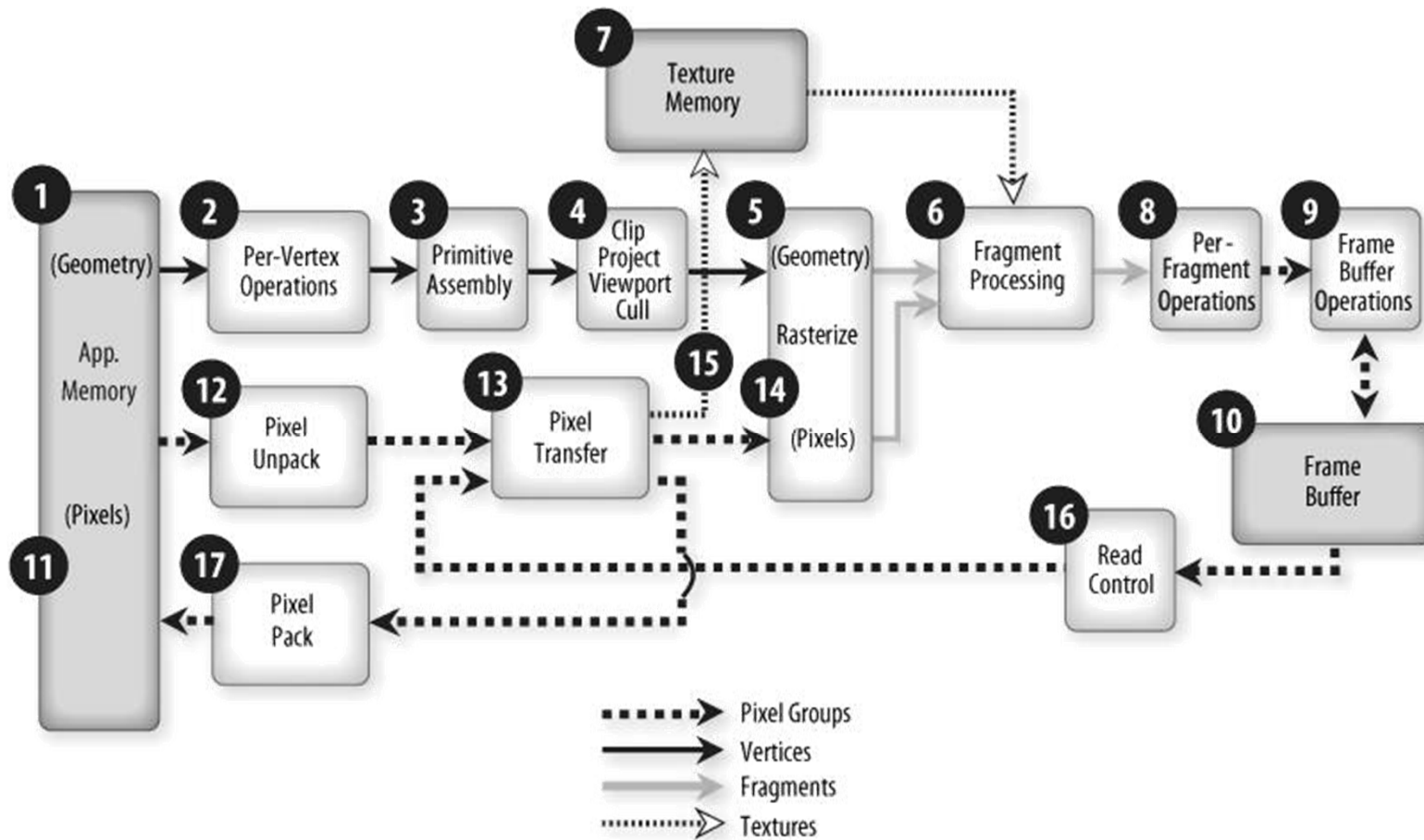
- Es modifiquen els buffers que s'hagin escollit amb `glDrawBuffers`
- Es veu afectada per `glColorMask`, `glDepthMask`...

Handwritten code in red:

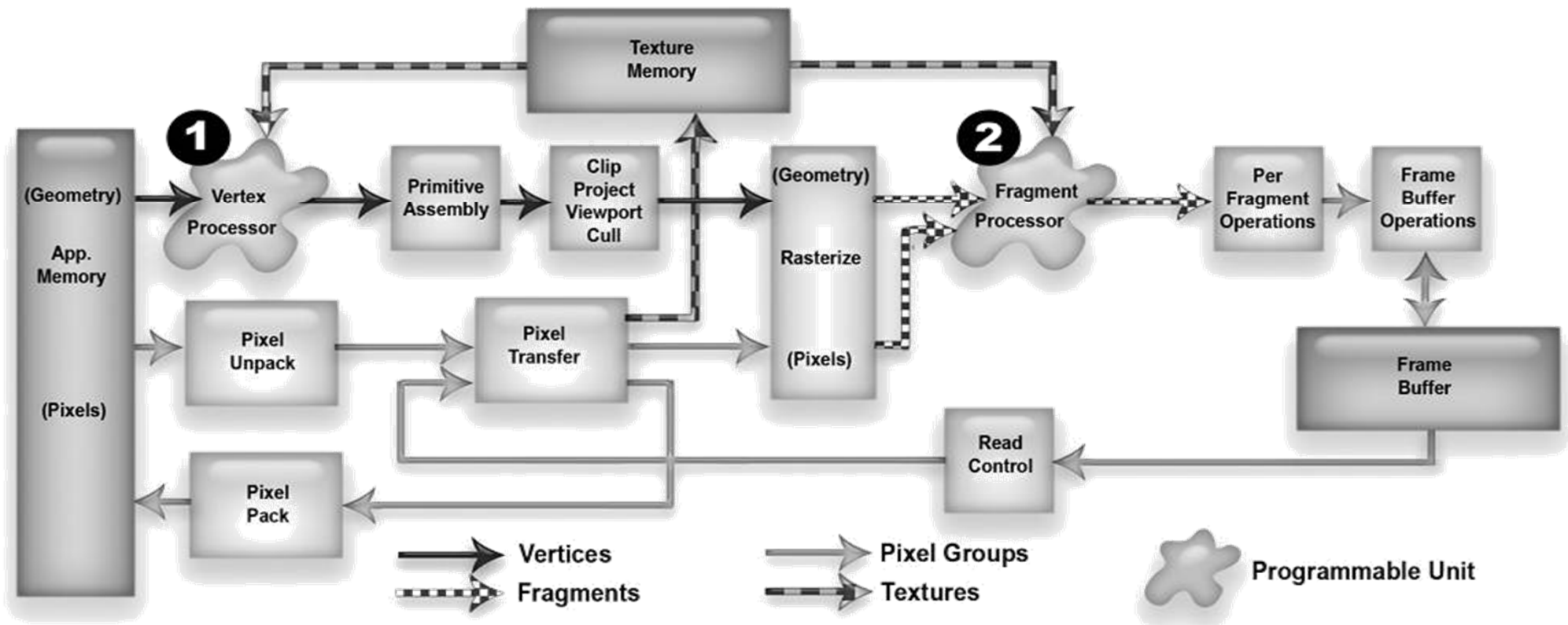
```
glColorMask(GL_TRUE, R1, G1, B1, A1)  
glDepthMask(GL_TRUE)
```

PIPELINE PROGRAMMABLE

Pipeline fix...



i pipeline programable



Pipeline programable

- **Vertex processor:** part de la GPU capaç d'executar un programa per cada vèrtex.
- **Fragment processor:** part de la GPU capaç d'executar un programa per cada fragment.
- **Shader:** codi font d'un programa (o part) per la GPU
 - Vertex shader, fragment shader, geometry shader
- **Program:** executable d'un programa per la GPU
 - Vertex program, fragment program, geometry program

Pipeline programable

- Quan s'activa un **vertex program**, en lloc d'executar-se les operacions **per-vèrtex** prefixades d'OpenGL, s'executa el vertex program.
- Quan s'activa un **fragment program**, en lloc d'executar-se les operacions **per-fragment** prefixades d'OpenGL, s'executa el fragment program.
- Normalment el vertex/fragment program més senzill haurà de reproduir part de la funcionalitat fixa d'OpenGL.